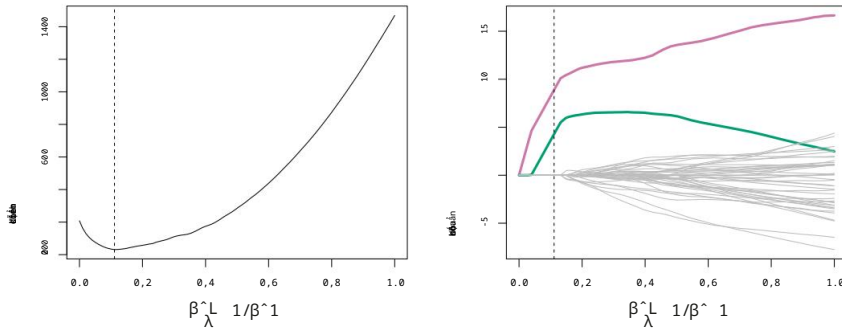




HÌNH 6.13. Bên trái: MSE kiểm định chéo 10 lần cho thuật toán lasso, được áp dụng cho Bộ dữ liệu mô phỏng thưa thớt từ Hình 6.9. Bên phải: Vùng lasso tương ứng. Các ước tính hệ số được hiển thị. Hai biến tín hiệu được hiển thị bằng màu sắc. và các biến nhiễu được hiển thị bằng màu xám. Các đường chấm dọc biểu thị vùng chọn (lasso) phù hợp nhất với sai số kiểm định chéo nhỏ nhất.

các quan sát. Ngược lại, giải pháp bình phương tối thiểu—được hiển thị ở phía xa Phần bên phải của bảng bên phải trong Hình 6.13—gán một hệ số lớn Ước tính chỉ dựa trên một trong hai biến tín hiệu.

6.3 Các phương pháp giảm chiều dữ liệu

Các phương pháp mà chúng ta đã thảo luận cho đến nay trong chương này đã kiểm soát Phương sai có thể được xác định theo hai cách khác nhau, hoặc bằng cách sử dụng một tập hợp con của các biến ban đầu, hoặc bằng cách thu hẹp hệ số của chúng về 0. Tất cả các phương pháp này đều... được định nghĩa bằng cách sử dụng các biến dự đoán ban đầu, $X_1, X_2, \dots, X_p$. Bây giờ chúng ta sẽ khám phá một nhóm các phương pháp biến đổi các biến dự đoán và sau đó điều chỉnh ít nhất Mô hình bình phương sử dụng các biến đã được biến đổi. Chúng ta sẽ gọi những kỹ thuật này là các phương pháp giảm chiều.

Gọi $Z_1, Z_2, \dots, Z_M$ là $M < p$ tổ hợp tuyến tính của các tổ hợp ban đầu của chúng ta. p biến dự báo. Tức là,

$$Z_m = \sum_{j=1}^p \varphi_{jm} X_j \quad (6.16)$$

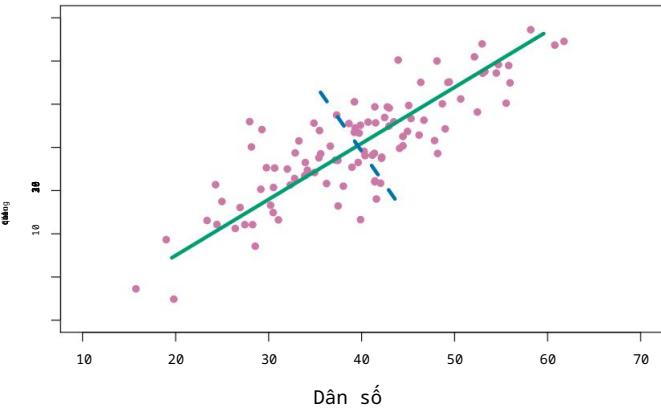
với một số hằng số $\varphi_{1m}, \varphi_{2m}, \dots, \varphi_{pm}, m=1, \dots, M$. Sau đó, chúng ta có thể khớp mô hình hồi quy tuyến tính

$$y_i = \theta_0 + \sum_{m=1}^M \theta_{zm} z_{im} + \epsilon_i, i=1, \dots, n, \quad (6.17)$$

sử dụng phương pháp bình phương nhỏ nhất. Lưu ý rằng trong (6.17), các hệ số hồi quy được đưa ra bởi $\theta_0, \theta_1, \dots, \theta_M$. Nếu các hằng số $\varphi_{1m}, \varphi_{2m}, \dots, \varphi_{pm}$ được chọn một cách khôn ngoan, thì Các phương pháp giảm chiều dữ liệu như vậy thường cho kết quả tốt hơn phương pháp bình phương tối thiểu hồi quy. Nói cách khác, việc khớp (6.17) bằng phương pháp bình phương nhỏ nhất có thể dẫn đến kết quả tốt hơn so với việc lắp đặt (6.1) bằng phương pháp bình phương nhỏ nhất.

Thuật ngữ giảm chiều xuất phát từ thực tế rằng phương pháp này giảm bớt vấn đề ước lượng $p+1$ hệ số $\beta_0, \beta_1, \dots, \beta_p$ xuống còn...

kích thước
sự giảm bớt
tuyến tính
sự kết hợp



HÌNH 6.14. Quy mô dân số (pop) và chỉ tiêu quảng cáo (ad) cho 100 doanh nghiệp khác nhau. Các thành phố được biểu thị bằng các vòng tròn màu tím. Đường kẻ liền màu xanh lá cây biểu thị thành phố chính đầu tiên, thành phần, và đường chấm xanh biểu thị thành phần chính thứ hai.

Bài toán đơn giản hơn là ước lượng $M + 1$ hệ số $\theta_0, \theta_1, \dots, \theta_M$, trong đó $M < p$. Nói cách khác, kích thước của vấn đề đã được thu nhỏ lại từ $p + 1$ đến $M + 1$.

Lưu ý rằng từ (6.16),

$$\theta_{zm} = \sum_{m=1}^M \theta_m \varphi_{mxij} = \sum_{j=1}^p \theta_m \varphi_{mxij} = \sum_{j=1}^p \beta_{jxij},$$

Ở đâu

$$\beta_j = \sum_{m=1}^M \theta_m \varphi_{jm}. \tag{6.18}$$

Do đó (6.17) có thể được coi là trường hợp đặc biệt của tuyến tính ban đầu mô hình hồi quy được đưa ra bởi (6.1). Giảm chiều giúp hạn chế các hệ số β_j ước tính, vì bây giờ chúng phải có dạng (6.18). Sự ràng buộc về hình thức của các hệ số này có khả năng gây sai lệch ước tính hệ số. Tuy nhiên, trong trường hợp p lớn hơn nhiều so với n , Việc lựa chọn giá trị $M < p$ có thể làm giảm đáng kể phương sai của mô hình phù hợp hệ số. Nếu $M = p$ và tất cả các Z_m đều độc lập tuyến tính thì (6.18) không đặt ra bất kỳ ràng buộc nào. Trong trường hợp này, không có sự giảm chiều nào xảy ra, và do đó việc lắp ráp (6.17) tương đương với việc thực hiện bình phương nhỏ nhất trên p ban đầu các yếu tố dự báo.

Tất cả các phương pháp giảm chiều đều hoạt động theo hai bước. Đầu tiên, các biến dự báo đã được biến đổi $Z_1, Z_2, \dots, Z_M$ được thu được. Thứ hai, mô hình được điều chỉnh sử dụng các biến dự báo M này. Tuy nhiên, việc lựa chọn $Z_1, Z_2, \dots, Z_M$, hay nói cách khác, việc lựa chọn các φ_{jm} , có thể được thực hiện theo nhiều cách khác nhau. Trong trường hợp này Trong chương này, chúng ta sẽ xem xét hai phương pháp tiếp cận cho nhiệm vụ này: các thành phần chính và phương pháp bình phương tối thiểu từng phần.

6.3.1 Hồi quy thành phần chính

Phân tích thành phần chính (PCA) là một phương pháp phổ biến để thu được kết quả.

hiệu trưởng
các thành phần
Phân tích

PCA là một tập hợp các đặc trưng có chiều thấp được rút gọn từ một tập hợp lớn các biến. PCA được thảo luận chi tiết hơn như một công cụ cho học không giám sát trong Chương 12. Ở đây, chúng tôi mô tả việc sử dụng nó như một kỹ thuật giảm chiều dữ liệu cho hồi quy.

Tổng quan về phân tích thành phần chính

PCA là một kỹ thuật để giảm kích thước của ma trận dữ liệu X có kích thước $n \times p$. Hướng thành phần chính thứ nhất của dữ liệu là hướng mà các quan sát biến thiên nhiều nhất. Ví dụ, hãy xem Hình 6.14, hiển thị quy mô dân số (pop) tính bằng hàng chục nghìn người và chỉ tiêu quảng cáo cho một công ty cụ thể (ad) tính bằng nghìn đô la, cho 100 thành phố. Đường liền màu xanh lá cây biểu thị hướng thành phần chính thứ nhất của dữ liệu. Chúng ta có thể thấy bằng mắt thường rằng đây là hướng mà dữ liệu có sự biến thiên lớn nhất. Nghĩa là, nếu chúng ta chiếu 100 quan sát lên đường thẳng này (như được hiển thị trong bảng bên trái của Hình 6.15), thì các quan sát được chiếu sẽ có phương sai lớn nhất có thể; chiếu các quan sát lên bất kỳ đường thẳng nào khác sẽ cho ra các quan sát được chiếu có phương sai thấp hơn. Việc chiếu một điểm lên một đường thẳng chỉ đơn giản là tìm vị trí trên đường thẳng gần điểm đó nhất.

Thành phần chính thứ nhất được hiển thị bằng đồ thị trong Hình 6.14, nhưng Nó có thể được tóm tắt bằng toán học như thế nào? Nó được biểu thị bằng công thức sau:

$$Z_1 = 0,839 \times (pop - \overline{pop}) + 0,544 \times (ad - \overline{ad}). \tag{6.19}$$

Ở đây $\phi_1 = 0,839$ và $\phi_2 = 0,544$ là các hệ số tải thành phần chính, xác định hướng được đề cập ở trên. Trong (6.19), pop biểu thị giá trị trung bình của tất cả các giá trị pop trong tập dữ liệu này, và ad biểu thị giá trị trung bình của tất cả chỉ tiêu quảng cáo. Ý tưởng là trong mọi tổ hợp tuyến tính có thể có của pop và ad sao cho $\phi_1^2 + \phi_2^2 = 1$, tổ hợp tuyến tính cụ thể này mang lại phương sai cực đại. Tức là đây là tổ hợp tuyến tính mà $Var(\phi_1 \times (pop - \overline{pop}) + \phi_2 \times (ad - \overline{ad}))$ được tối đa hóa. Cần phải chỉ xem xét các tổ hợp tuyến tính có dạng $\phi_1 (pop - \overline{pop}) + \phi_2 (ad - \overline{ad}) = 1$, vì nếu không chúng ta có thể tăng ϕ_1 và ϕ_2 một cách tùy ý để làm tăng phương sai.

Trong (6.19), cả hai hệ số tải đều dương và có kích thước tương tự nhau, do đó Z_1 gần như là giá trị trung bình của hai biến.

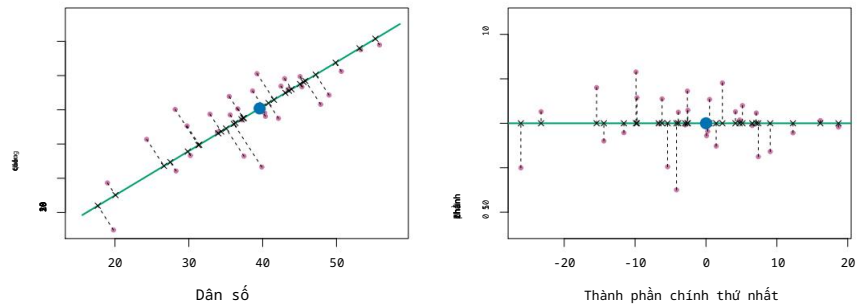
Vì $n = 100$, pop và ad là các vectơ có độ dài 100, và Z_1 trong (6.19) cũng vậy. Ví dụ,

$$z_{11} = 0,839 \times (pop_1 - \overline{pop}) + 0,544 \times (ad_1 - \overline{ad}). \tag{6.20}$$

Các giá trị $z_{11}, \dots, z_{n1}$ được gọi là điểm số thành phần chính và có thể được thấy ở bảng bên phải của Hình 6.15.

Ngoài ra còn có một cách diễn giải khác về PCA: vectơ thành phần chính đầu tiên xác định đường thẳng gần nhất với dữ liệu. Ví dụ, trong Hình 6.14, đường thẳng thành phần chính đầu tiên tối thiểu hóa tổng bình phương khoảng cách vuông góc giữa mỗi điểm và đường thẳng. Các khoảng cách này được biểu diễn bằng các đoạn thẳng đứt nét ở phía bên trái.

6. Bộ dữ liệu này khác với dữ liệu **Quảng cáo** được thảo luận trong Chương 3.



HÌNH 6.15. Một phần dữ liệu quảng cáo. Dân số trung bình và ngân sách quảng cáo được biểu thị bằng một vòng tròn màu xanh lam. Bên trái: Hướng của thành phần chính thứ nhất là được hiển thị bằng màu xanh lá cây. Đó là chiều mà dữ liệu thay đổi nhiều nhất, và nó cũng Xác định đường thẳng gần nhất với tất cả n điểm quan sát. Khoảng cách từ mỗi Các quan sát đối với thành phần chính được biểu diễn bằng đường nét đứt màu đen. Các phân đoạn. Chấm màu xanh lam biểu thị (pop, ad). Bên phải: Bảng bên trái đã được Đã xoay sao cho hướng của thành phần chính thứ nhất trùng với trục x.

bảng của Hình 6.15, trong đó các dấu thập biểu thị hình chiếu của mỗi chỉ vào đường thành phần chính thứ nhất. Thành phần chính thứ nhất đã được chọn sao cho các quan sát dự kiến càng gần nhau càng tốt. so sánh với những quan sát ban đầu.

Trong bảng bên phải của Hình 6.15, bảng bên trái đã được được xoay sao cho hướng của thành phần chính thứ nhất trùng với Trục x. Có thể chứng minh rằng điểm số thành phần chính đầu tiên cho quan sát thứ i, được đưa ra trong (6.20), là khoảng cách theo hướng x của dấu chéo thứ i tính từ số 0. Ví dụ, điểm ở góc dưới bên trái của Bảng bên trái của Hình 6.15 có thành phần chính âm lớn. điểm số, z11 = 26,1, trong khi điểm ở góc trên bên phải có giá trị lớn điểm số dương, z11 = 18,7. Các điểm số này có thể được tính toán trực tiếp bằng cách sử dụng (6.20).

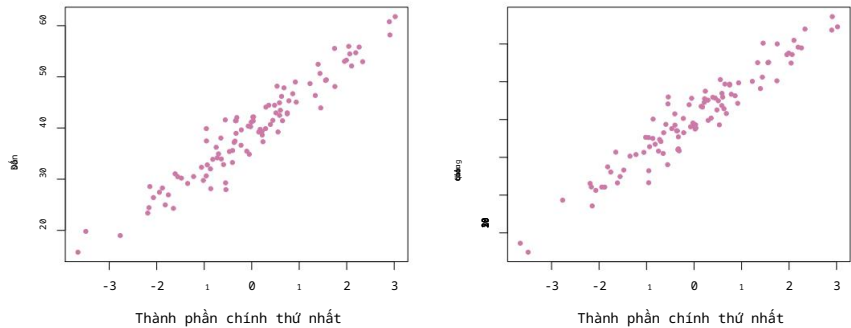
Ta có thể coi các giá trị của thành phần chính Z1 như là tóm tắt bằng một con số duy nhất về ngân sách chung cho dân số và quảng cáo tại mỗi địa điểm.

ví dụ này, nếu $z_{11} = 0.839 \times (\text{pop}_i - \text{pop}) + 0.544 \times (\text{ad}_i - \text{ad}) < 0$, Điều này cho thấy một thành phố có quy mô dân số dưới mức trung bình và chi tiêu quảng cáo dưới mức trung bình. Điểm số tích cực cho thấy điều ngược lại. Một thành phố có thể làm tốt đến mức nào?

Liệu một con số duy nhất có thể đại diện cho cả số lượng người xem (pop) và số lượng quảng cáo (ad) không? Trong trường hợp này, Hình 6.14 cho thấy điều đó. rằng nhạc pop và nhạc quảng cáo có mối quan hệ gần như tuyến tính, và vì vậy chúng ta có thể Dự kiến rằng một bản tóm tắt bằng một con số duy nhất sẽ hoạt động tốt. Hình 6.16 hiển thị z11 so sánh với cả nhạc pop và nhạc quảng cáo. Các đồ thị cho thấy mối quan hệ chặt chẽ giữa thành phần chính đầu tiên và hai đặc điểm. Nói cách khác, thành phần đầu tiên Phân tích thành phần chính dường như nắm bắt được hầu hết thông tin chứa đựng trong các công cụ dự đoán nhạc pop và quảng cáo.

Cho đến nay, chúng ta đã tập trung vào thành phần chính thứ nhất. Nói chung, người ta có thể xây dựng tối đa p thành phần chính khác nhau. Thành phần chính thứ hai

7. Các thành phần chính được tính toán sau khi chuẩn hóa cả nhạc pop và nhạc quảng cáo. một cách tiếp cận phổ biến. Do đó, trục x trên Hình 6.15 và 6.16 không nằm trên cùng một mặt phẳng tỉ lệ.



HÌNH 6.16. Đồ thị biểu diễn điểm số thành phần chính thứ nhất z_1 so với pop và ad . Các mối quan hệ rất bền chặt.

Thành phần chính Z_2 là tổ hợp tuyến tính của các biến không tương quan với Z_1 , và có phương sai lớn nhất theo ràng buộc này.

Hướng của thành phần chính thứ hai được minh họa bằng đường chấm xanh lam trong Hình 6.14. Kết quả cho thấy điều kiện tương quan bằng không giữa Z_1 và Z_2 .

Điều này tương đương với điều kiện hướng phải vuông góc, hay trực giao, với hướng thành phần chính thứ nhất. Thành phần chính thứ hai

vuông góc-

thành phần được cho bởi công thức

trực giao

$$Z_2 = 0,544 \times (pop - \overline{pop}) - 0,839 \times (ad - \overline{ad}).$$

Vì dữ liệu quảng cáo có hai yếu tố dự báo, nên hai thành phần chính đầu tiên chứa tất cả thông tin có trong pop và ad . Tuy nhiên, bằng cách

Trong cấu trúc này, thành phần đầu tiên sẽ chứa nhiều thông tin nhất. Ví dụ, hãy xem xét sự biến thiên lớn hơn nhiều của z_1 (trục x) so với

z_2 (trục y) trong bảng bên phải của Hình 6.15. Thực tế là

Điểm số thành phần chính thứ hai gần bằng 0 hơn nhiều cho thấy rằng

Thành phần này thu thập được ít thông tin hơn nhiều. Để minh họa thêm, Hình 6.17 hiển thị z_2 so với pop và ad . Có rất ít mối quan hệ giữa chúng.

thành phần chính thứ hai và hai biến dự báo này, một lần nữa cho thấy

Trong trường hợp này, người ta chỉ cần thành phần chính đầu tiên để

Thể hiện chính xác ngân sách **quảng cáo** và **nhạc pop**.

Với dữ liệu hai chiều, như trong ví dụ quảng cáo của chúng ta, chúng ta có thể xây dựng tối đa hai thành phần chính. Tuy nhiên, nếu chúng ta có các thành phần khác thì sẽ không có.

các yếu tố dự báo, chẳng hạn như độ tuổi dân số, mức thu nhập, trình độ học vấn, v.v.

sau đó có thể chế tạo thêm các bộ phận khác. Chúng sẽ lần lượt được chế tạo.

tối đa hóa phương sai, với điều kiện là không tương quan với

các thành phần trước đó.

Phương pháp hồi quy thành phần chính

Phương pháp hồi quy thành phần chính (PCR) bao gồm việc xây dựng M thành phần chính đầu tiên, $Z_1, \dots, Z_M$, và sau đó sử dụng chúng.

hiệu trưởng

các thành phần

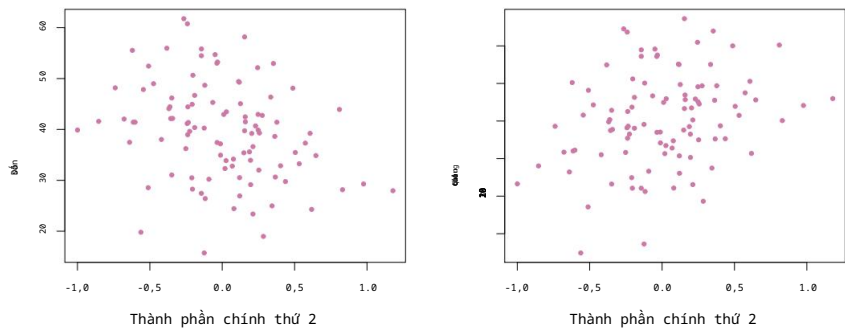
hồi quy

các thành phần được sử dụng làm biến dự báo trong mô hình hồi quy tuyến tính được xây dựng

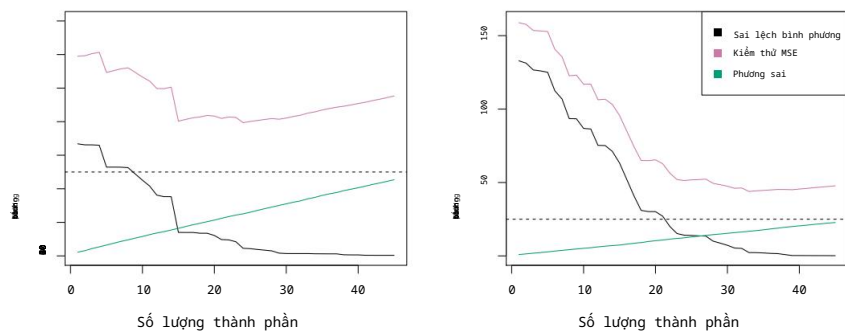
bằng phương pháp bình phương nhỏ nhất. Ý tưởng chính là thường một số ít các thành phần chính.

Các thành phần này đủ để giải thích hầu hết sự biến thiên trong dữ liệu, cũng như

như mối quan hệ với phản hồi. Nói cách khác, chúng ta giả định rằng



HÌNH 6.17. Đồ thị biểu diễn điểm số thành phần chính thứ hai z_2 so với **pop** và **quảng cáo**. Mỗi quan hệ giữa các bên rất yếu.

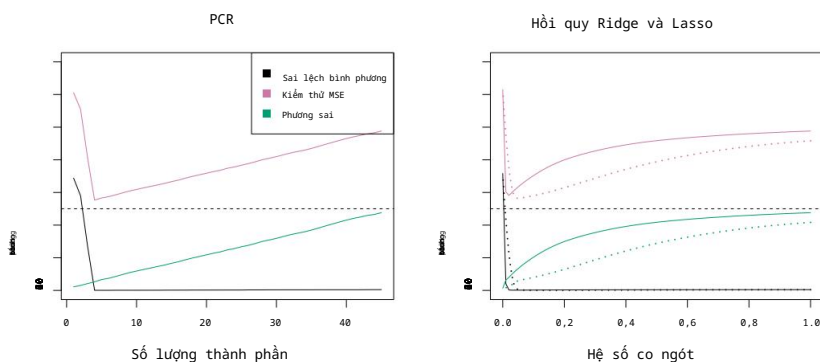


HÌNH 6.18. Phương pháp PCR được áp dụng cho hai bộ dữ liệu mô phỏng. Trong mỗi bảng, Đường chấm ngang biểu thị sai số không thể giảm thiểu. Bên trái: Dữ liệu mô phỏng từ Hình 6.8. Bên phải: Dữ liệu mô phỏng từ Hình 6.9.

Các hướng mà $X_1, \dots, X_p$ thể hiện sự biến thiên lớn nhất là các hướng có liên quan đến Y . Mặc dù giả định này không được đảm bảo là đúng. Đúng vậy, nó thường tỏ ra là một phép xấp xỉ đủ hợp lý để cung cấp Kết quả tốt.

Nếu giả định cơ bản của PCR là đúng, thì việc khớp dữ liệu bằng phương pháp bình phương tối thiểu sẽ... Việc áp dụng mô hình cho $Z_1, \dots, Z_M$ sẽ cho kết quả tốt hơn so với việc sử dụng phương pháp bình phương tối thiểu. mô hình thành $X_1, \dots, X_p$, vì hầu hết hoặc tất cả thông tin trong dữ liệu đó liên quan đến phản hồi được chứa trong $Z_1, \dots, Z_M$, và bằng cách chỉ ước tính Các hệ số M p có thể giúp giảm thiểu hiện tượng quá khớp. Trong dữ liệu quảng cáo, Thành phần chính đầu tiên giải thích phần lớn sự biến thiên trong cả **nhạc pop** và **nhạc quảng cáo**. Vì vậy, một phương pháp hồi quy thành phần chính sử dụng biến đơn này để dự đoán Một số phản hồi tích cực, chẳng hạn như **doanh số bán hàng**, có khả năng sẽ đạt kết quả khá tốt.

Hình 6.18 hiển thị sự phù hợp của PCR trên các tập dữ liệu mô phỏng từ Hình 6.8 và 6.9. Lưu ý rằng cả hai tập dữ liệu đều được tạo ra bằng cách sử dụng $n = 50$. các quan sát và $p = 45$ biến dự đoán. Tuy nhiên, trong khi phản hồi trong Tập dữ liệu đầu tiên là một hàm của tất cả các biến dự đoán, còn phản hồi trong tập dữ liệu thứ hai được tạo ra chỉ bằng cách sử dụng hai trong số các biến dự đoán. Các đường cong là... được vẽ dưới dạng hàm của M , số lượng thành phần chính được sử dụng làm các biến dự báo trong mô hình hồi quy. Khi sử dụng nhiều thành phần chính hơn.



HÌNH 6.19. Phương pháp PCR, hồi quy Ridge và Lasso được áp dụng cho một tập dữ liệu mô phỏng trong đó năm thành phần chính đầu tiên của X chứa tất cả các thành phần chính.

thông tin về phản hồi Y . Trong mỗi bảng, sai số không thể giảm $\text{Var}()$ là

Được thể hiện bằng một đường chấm ngang. Bên trái: Kết quả PCR. Bên phải: Kết quả lasso.

(đường liền nét) và hồi quy Ridge (đường chấm). Trục x hiển thị hệ số co rút.

của các ước lượng hệ số, được định nghĩa là chuẩn 2 của hệ số bị thu hẹp

ước tính chia cho chuẩn 2 của ước tính bình phương nhỏ nhất.

Trong mô hình hồi quy, độ lệch giảm nhưng phương sai tăng. Điều này

Kết quả cho thấy sai số bình phương trung bình có dạng hình chữ U điển hình. Khi $M = p = 45$,

Sau đó, lượng PCR được tính toán đơn giản bằng phương pháp bình phương tối thiểu sử dụng tất cả các dữ liệu ban đầu.

các yếu tố dự báo. Hình minh họa cho thấy việc thực hiện PCR với một phương pháp thích hợp

Việc lựa chọn M có thể mang lại sự cải thiện đáng kể so với phương pháp bình phương tối thiểu,

đặc biệt là ở bảng bên trái. Tuy nhiên, khi xem xét hồi quy Ridge, ta thấy rằng...

Và dựa trên kết quả của phương pháp lasso trong Hình 6.5, 6.8 và 6.9, chúng ta thấy rằng PCR không có hiệu quả tương đương với hai phương pháp co ngót trong ví dụ này.

Hiệu suất PCR tương đối kém hơn trong Hình 6.18 là một hệ quả.

Điều này xuất phát từ thực tế là dữ liệu được tạo ra theo cách đòi hỏi nhiều thành phần chính để có thể mô hình hóa phản hồi một cách đầy đủ.

Ngược lại, PCR sẽ có hiệu quả tốt hơn trong trường hợp một vài xét nghiệm chính đầu tiên được thực hiện.

Các thành phần này đủ để nắm bắt hầu hết sự biến thiên trong các yếu tố dự báo.

Cũng như mối quan hệ với phản hồi. Bảng bên trái của Hình 6.19 minh họa kết quả từ một bộ dữ liệu mô phỏng khác được thiết kế để

sẽ thuận lợi hơn cho PCR. Ở đây, phản ứng được tạo ra theo cách như vậy.

Điều đó phụ thuộc hoàn toàn vào năm thành phần chính đầu tiên. Bây giờ thì

Sai lệch giảm nhanh chóng xuống bằng không khi M , số lượng thành phần chính được sử dụng, tăng lên.

Trong PCR, giá trị này tăng lên. Sai số bình phương trung bình hiển thị mức tối thiểu rõ ràng tại

$M = 5$. Bảng bên phải của Hình 6.19 hiển thị kết quả về những điều này.

dữ liệu sử dụng hồi quy Ridge và Lasso. Cả ba phương pháp đều mang lại sự cải thiện đáng kể so

với phương pháp bình phương tối thiểu. Tuy nhiên, PCR và hồi quy Ridge

Hiệu quả hơn một chút so với đây thông lượng.

Chúng tôi lưu ý rằng mặc dù PCR cung cấp một cách đơn giản để thực hiện hồi quy bằng cách sử

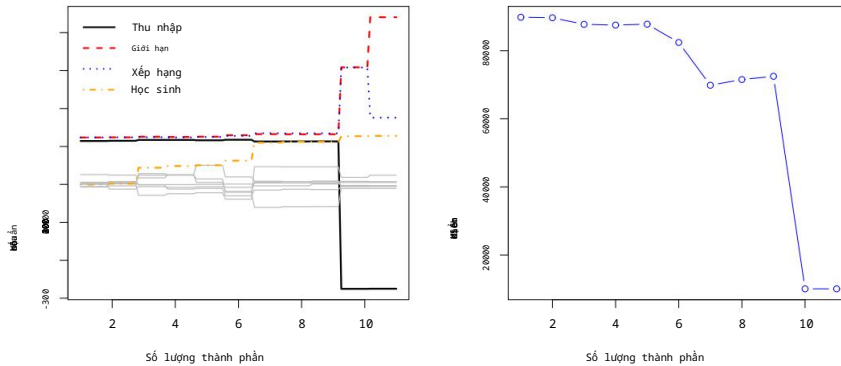
dụng $M < p$ biến dự đoán, nhưng nó không phải là một phương pháp lựa chọn đặc trưng. Điều này

là vì mỗi trong số M thành phần chính được sử dụng trong phép hồi quy đều là một

tổ hợp tuyến tính của tất cả p trong số các đặc điểm ban đầu. Ví dụ, trong (6.19),

Z_1 là sự kết hợp tuyến tính của cả pop và ad . Do đó, mặc dù PCR thường hoạt động khá tốt trong

nhiều trường hợp thực tế, nhưng nó không mang lại kết quả như mong muốn.



HÌNH 6.20. Bên trái: Ước tính hệ số chuẩn hóa PCR trên dữ liệu **Credit** .

được thiết lập cho các giá trị khác nhau của M . Bên phải: Sai số bình phương trung bình (MSE) thu được từ phương pháp kiểm chứng chéo mười lần. sử dụng PCR, như một hàm của M .

Phát triển một mô hình dựa trên một tập hợp nhỏ các tính năng ban đầu.

Theo nghĩa này, PCR có mối liên hệ mật thiết hơn với phương pháp hồi quy Ridge so với... lasso. Trên thực tế, người ta có thể chứng minh rằng PCR và hồi quy Ridge rất gần nhau. có liên quan. Thậm chí người ta có thể coi hồi quy Ridge như một phiên bản liên tục của PCR!⁸

Trong PCR, số lượng thành phần chính, M , thường được chọn bởi Kiểm định chéo. Kết quả của việc áp dụng PCR cho tập dữ liệu **Tín dụng** là: Như thể hiện trong Hình 6.20; bảng bên phải hiển thị các lỗi kiểm định chéo thu được, theo hàm của M . Trên dữ liệu này, lỗi kiểm định chéo thấp nhất Lỗi xảy ra khi có $M = 10$ thành phần; điều này tương ứng với việc hầu như không giảm kích thước nào cả, vì PCR với $M = 11$ là tương đương.

Đơn giản chỉ là thực hiện phương pháp bình phương tối thiểu.

Khi thực hiện PCR, chúng tôi thường khuyên nên chuẩn hóa từng yếu tố dự báo, sử dụng (6.6), trước khi tạo ra các thành phần chính. Việc chuẩn hóa này đảm bảo rằng tất cả các biến đều nằm trên cùng một thang đo. Trong trường hợp không có

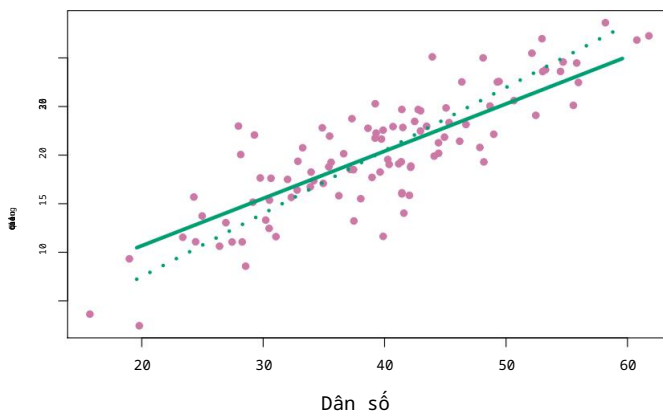
Trong quá trình chuẩn hóa, các biến có phương sai cao sẽ có xu hướng đóng vai trò lớn hơn. Vai trò của nó trong các thành phần chính thu được, và thang đo mà các biến được sử dụng cuối cùng sẽ ảnh hưởng đến mô hình PCR cuối cùng.

Tuy nhiên, nếu tất cả các biến số đều được đo bằng cùng một đơn vị (ví dụ: kilogram, hoặc inch), thì người ta có thể chọn không tiêu chuẩn hóa chúng.

6.3.2 Phương pháp bình phương tối thiểu từng phần

Phương pháp PCR mà chúng ta vừa mô tả bao gồm việc xác định các tổ hợp tuyến tính, hay các hướng, thể hiện tốt nhất các biến dự đoán $X_1, \dots, X_p$. Những tổ hợp này Các hướng được xác định một cách không giám sát, vì phản hồi Y là không được sử dụng để giúp xác định hướng của các thành phần chính. Tức là, Phản hồi này không giám sát việc xác định các thành phần chính. Do đó, PCR có một nhược điểm: không có sự đảm bảo.

⁸Thông tin chi tiết hơn có thể được tìm thấy trong Mục 3.5 của cuốn sách "Các yếu tố của học máy thống kê" của tác giả. Hastie, Tibshirani và Friedman.



HÌNH 6.21. Đối với dữ liệu quảng cáo, hướng PLS đầu tiên (đường liền nét) và hướng PCR đầu tiên (đường chấm) được hiển thị.

Những hướng dẫn giải thích rõ nhất các yếu tố dự báo cũng sẽ là những hướng dẫn tốt nhất.

Hướng dẫn sử dụng để dự đoán phản hồi. Các phương pháp không giám sát là

Sẽ được thảo luận chi tiết hơn trong Chương 12.

Chúng tôi xin giới thiệu phương pháp bình phương tối thiểu từng phần (PLS), một phương pháp thay thế có giám sát cho PCR. Giống như PCR, PLS là một phương pháp giảm chiều dữ liệu, trước tiên xác định một tập hợp các tính năng mới $Z_1, \dots, Z_M$ là sự kết hợp tuyến tính của các tính năng ban đầu.

các đặc điểm, và sau đó áp dụng mô hình tuyến tính thông qua phương pháp bình phương tối thiểu bằng cách sử dụng M đặc điểm mới này. nhưng không giống như PCR, PLS xác định các đặc điểm mới này trong một mô hình có giám sát. cách này-nghĩa là, nó sử dụng phản hồi Y để xác định những điều mới

các tính năng không chỉ mô phỏng tốt các tính năng cũ mà còn ...

liên quan đến phản hồi. Nói một cách khái quát, phương pháp PLS cố gắng

Tìm hướng dẫn giúp giải thích cả phản hồi và các yếu tố dự báo.

Chúng ta sẽ mô tả cách tính toán hướng PLS đầu tiên. Sau khi chuẩn hóa các biến dự đoán p , PLS tính toán hướng đầu tiên Z_1 bằng cách thiết lập

mỗi ϕ_{j1} trong (6.16) bằng hệ số tử hồi quy tuyến tính đơn giản

của Y lên X_j . Có thể chứng minh rằng hệ số này tỷ lệ thuận với hệ số tương quan giữa Y và X_j . Do đó, khi tính toán $Z_1 = \phi_{j1}X_j$, PLS

đặt trọng số cao nhất cho các biến có mối liên hệ chặt chẽ nhất với

câu trả lời.

Hình 6.21 hiển thị một ví dụ về PLS trên tập dữ liệu tổng hợp với Sales

ở mỗi trong số 100 khu vực là biến phản hồi, và hai biến dự báo; Quy mô dân số

và Chi tiêu Quảng cáo. Đường màu xanh lá cây liền nét biểu thị hướng PLS đầu tiên, trong khi đường chấm chấm thể hiện hướng thành phần chính đầu tiên.

PLS đã chọn một hướng đi có sự thay đổi ít nhất về kích thước quảng cáo trên mỗi đơn vị.

sự thay đổi đơn vị trong chiều dân số, so với PCA. Điều này cho thấy rằng dân số

có mối tương quan cao hơn với phản hồi so với quảng cáo. Hướng PLS

Nó không khớp với các biến dự đoán chính xác như PCA, nhưng nó thực hiện công việc tốt hơn.

Giải thích câu trả lời.

Để xác định hướng PLS thứ hai, trước tiên chúng ta điều chỉnh từng biến số.

Đối với Z_1 , bằng cách hồi quy từng biến trên Z_1 và lấy phần dư. Các phần dư này có thể được hiểu là thông tin còn lại chưa được xử lý.

được giải thích bởi hướng PLS đầu tiên. Sau đó, chúng ta tính toán Z_2 bằng cách sử dụng điều này hoặc-

ít nhất một phần
hình vuông

$j=1$

Dữ liệu được trực giao hóa theo cùng một cách thức như Z_1 đã được tính toán dựa trên trên dữ liệu gốc. Phương pháp lặp này có thể được lặp lại M lần để xác định nhiều thành phần PLS $Z_1, \dots, Z_M$. Cuối cùng, ở phần kết thúc này

Trong quy trình này, chúng tôi sử dụng phương pháp bình phương tối thiểu để khớp một mô hình tuyến tính nhằm dự đoán Y bằng cách sử dụng $Z_1, \dots, Z_M$ được thực hiện hoàn toàn giống như đối với PCR.

Tương tự như PCR, số lượng M các hướng bình phương tối thiểu từng phần được sử dụng trong PLS là một tham số điều chỉnh thường được lựa chọn bằng phương pháp kiểm định chéo. Chúng tôi nói chung, cần chuẩn hóa các biến dự đoán và biến phản hồi trước khi thực hiện PLS.

PLS phổ biến trong lĩnh vực hóa học lượng tử, nơi phát sinh nhiều biến số. từ các tín hiệu quang phổ số hóa. Trên thực tế, nó thường cho kết quả không tốt hơn. so với hồi quy Ridge hoặc PCR. Trong khi đó, việc giảm chiều có giảm sát Phương pháp PLS có thể giảm thiểu sai lệch, nhưng nó cũng có khả năng làm tăng phương sai, vì vậy Nhìn chung, lợi ích của PLS so với PCR là tương đương nhau.

6.4 Những cân nhắc trong không gian đa chiều

6.4.1 Dữ liệu đa chiều

Hầu hết các kỹ thuật thống kê truyền thống hồi quy và phân loại là Mục đích là dành cho thiết lập chiều thấp, trong đó n , số lượng quan sát, lớn hơn nhiều so với p , số lượng đặc trưng. Điều này là do...

Một phần là do trong phần lớn lịch sử của lĩnh vực này, phần lớn các vấn đề khoa học đòi hỏi sử dụng thống kê đều có chiều thấp.

Ví dụ, hãy xem xét nhiệm vụ phát triển một mô hình để dự đoán tình trạng của bệnh nhân. Huyết áp được xác định dựa trên tuổi tác, giới tính và chỉ số khối cơ thể của người đó. (BMI). Có ba biến dự báo, hoặc bốn nếu bao gồm cả hệ số chặn. mô hình, và có lẽ vài nghìn bệnh nhân có huyết áp và tuổi, giới tính và chỉ số BMI đều có sẵn. Do đó, n , và vì vậy vấn đề là Chiều thấp. (Ở đây, "chiều" được hiểu là kích thước của p .)

Trong 20 năm qua, các công nghệ mới đã thay đổi cách thức xử lý dữ liệu. Các khoản tiền này được thu thập trong nhiều lĩnh vực đa dạng như tài chính, tiếp thị và y học. Hiện nay, việc thu thập số lượng phép đo đặc trưng gần như không giới hạn (p rất lớn) đã trở nên phổ biến. Mặc dù p có thể cực kỳ lớn, nhưng số lượng Số lượng quan sát (n) thường bị hạn chế do chi phí, tính sẵn có của mẫu hoặc các lý do khác. những điều cần xem xét. Hai ví dụ như sau:

- 1. Thay vì dự đoán huyết áp chỉ dựa trên tuổi tác, giới tính, Ngoài chỉ số BMI, người ta cũng có thể thu thập các phép đo cho nửa triệu biến thể đơn nucleotide (SNP; đây là các đột biến DNA riêng lẻ tương đối phổ biến trong dân số) để đưa vào. mô hình dự đoán. Khi đó $n \approx 200$ và $p \approx 500.000$.
- 2. Một nhà phân tích tiếp thị quan tâm đến việc hiểu các mẫu hành vi mua sắm trực tuyến của người dùng có thể coi tất cả các cụm từ tìm kiếm được nhập vào như là các đặc điểm nổi bật. bởi người dùng của một công cụ tìm kiếm. Điều này đôi khi được gọi là mô hình "túi từ". Cùng một nhà nghiên cứu có thể có quyền truy cập vào kết quả tìm kiếm. lịch sử của chỉ vài trăm hoặc vài nghìn người dùng công cụ tìm kiếm. những người đã đồng ý chia sẻ thông tin của họ với nhà nghiên cứu. Đối với một người dùng nhất định, mỗi trong số p thuật ngữ tìm kiếm được chấm điểm là có mặt (0) hoặc

vắng mặt (1), tạo ra một vectơ đặc trưng nhị phân lớn. Sau đó $n \approx 1.000$ và p lớn hơn nhiều.

Các tập dữ liệu chứa nhiều đặc điểm hơn số quan sát thường được gọi là...
do có nhiều chiều. Các phương pháp cổ điển như hồi quy tuyến tính bình phương tối thiểu không phù hợp trong trường hợp này. Nhiều vấn đề phát sinh chiều cao
Các vấn đề trong phân tích dữ liệu đa chiều đã được thảo luận trước đó trong cuốn sách này. vì chúng cũng áp dụng khi $n > p$: điều này bao gồm vai trò của phương sai độ lệch sự đánh đổi và nguy cơ quá khớp. Mặc dù những vấn đề này luôn luôn liên quan, nhưng chúng có thể trở nên đặc biệt quan trọng khi số lượng đặc trưng tăng lên. Nó rất lớn so với số lượng quan sát.

Chúng ta đã định nghĩa thiết lập đa chiều là trường hợp số lượng đặc trưng p lớn hơn số lượng quan sát n . Nhưng những cân nhắc mà chúng ta sẽ thảo luận sau đây chắc chắn cũng áp dụng nếu p nhỏ hơn một chút. Nhỏ hơn n , và luôn cần ghi nhớ điều này khi thực hiện quá trình học tập có giám sát.

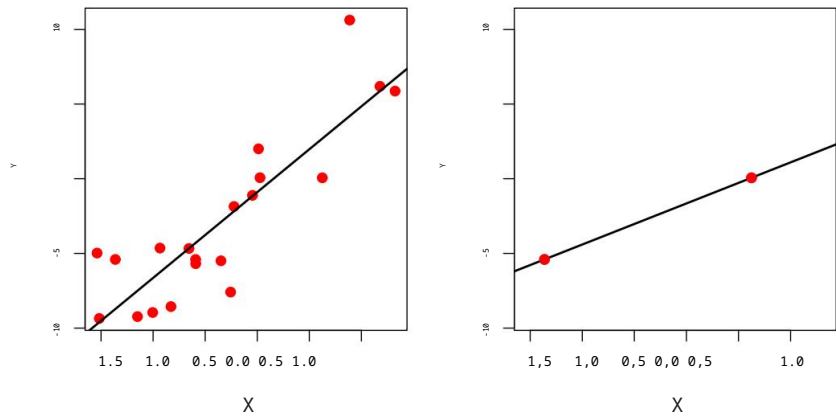
6.4.2 Điều gì xảy ra sai sót trong không gian đa chiều?

Để minh họa cho sự cần thiết phải chăm sóc đặc biệt và sử dụng các kỹ thuật chuyên biệt. Đối với hồi quy và phân loại khi $p > n$, chúng ta bắt đầu bằng cách xem xét điều gì Có thể xảy ra sai sót nếu chúng ta áp dụng một kỹ thuật thống kê không được thiết kế cho môi trường đa chiều. Vì mục đích này, chúng ta xem xét phương pháp hồi quy bình phương tối thiểu. Nhưng những khái niệm tương tự cũng áp dụng cho hồi quy logistic, phân tích phân biệt tuyến tính và các phương pháp thống kê cổ điển khác.

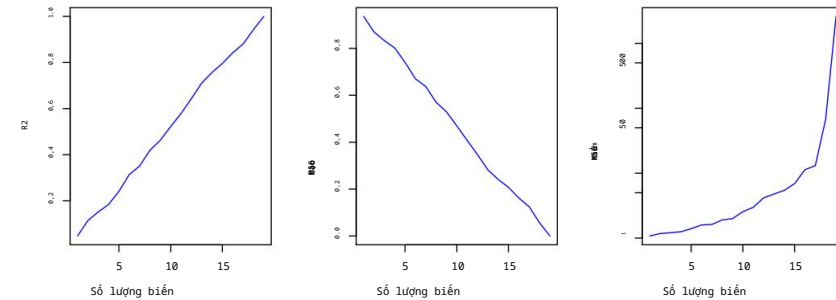
Khi số lượng đặc trưng p lớn bằng hoặc lớn hơn số lượng với n quan sát, phương pháp bình phương tối thiểu như mô tả trong Chương 3 không thể (hay nói đúng hơn, (Không nên) thực hiện. Lý do rất đơn giản: bất kể là hay không Thực tế không có mối liên hệ nào giữa các đặc điểm và phản hồi. Phương pháp bình phương tối thiểu sẽ cho ra một tập hợp các ước lượng hệ số dẫn đến kết quả hoàn hảo. Điều chỉnh sao cho phù hợp với dữ liệu, để phần dư bằng không.

Một ví dụ được minh họa trong Hình 6.22 với đặc điểm $p = 1$ (cộng thêm hệ số chặn). trong hai trường hợp: khi có 20 quan sát và khi chỉ có hai quan sát. Khi có 20 quan sát, $n > p$ và giá trị nhỏ nhất Đường hồi quy bình phương không hoàn toàn phù hợp với dữ liệu; thay vào đó, đường hồi quy Đường thẳng này nhằm mục đích xấp xỉ 20 quan sát một cách chính xác nhất có thể. Trên Mặt khác, khi chỉ có hai quan sát, thì bất kể Với các giá trị quan sát đó, đường hồi quy sẽ khớp chính xác với dữ liệu. Điều này gây ra vấn đề vì sự phù hợp hoàn hảo này gần như chắc chắn sẽ dẫn đến hiện tượng quá khớp dữ liệu. Nói cách khác, mặc dù có thể khớp hoàn hảo. dữ liệu huấn luyện trong môi trường đa chiều, mô hình tuyến tính thu được sẽ hoạt động cực kỳ kém trên một tập dữ liệu kiểm thử độc lập, và do đó Nó không phải là một mô hình hữu ích. Trên thực tế, chúng ta có thể thấy điều này đã xảy ra. Trong Hình 6.22: đường hồi quy bình phương tối thiểu thu được ở bảng bên phải sẽ Nó hoạt động rất kém trên một tập dữ liệu thử nghiệm bao gồm các quan sát ở bảng bên trái. Vấn đề rất đơn giản: khi $p > n$ hoặc $p \approx n$, một phép tính tối thiểu đơn giản Đường hồi quy bình phương quá linh hoạt và do đó dẫn đến hiện tượng quá khớp dữ liệu.

Hình 6.23 minh họa thêm rủi ro khi áp dụng phương pháp bình phương tối thiểu một cách thiếu cẩn trọng. khi số lượng đặc trưng p lớn. Dữ liệu được mô phỏng với $n = 20$ các quan sát và phép hồi quy được thực hiện với số lượng đặc trưng từ 1 đến 20.

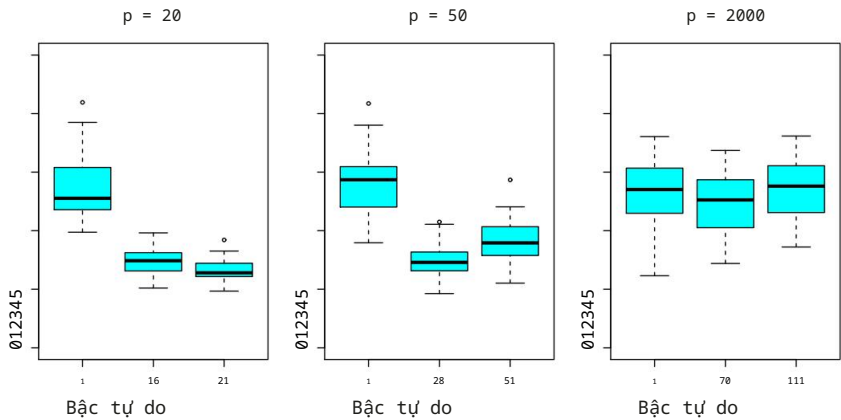


HÌNH 6.22. Bên trái: Hồi quy bình phương tối thiểu trong thiết lập chiều thấp. Bên phải: Hồi quy bình phương tối thiểu với $n = 2$ quan sát và hai tham số cần được xác định. ước tính (hệ số chặn và hệ số).



HÌNH 6.23. Trên một ví dụ mô phỏng với $n = 20$ quan sát huấn luyện, Các tính năng hoàn toàn không liên quan đến kết quả được thêm vào mô hình. Trái: Hệ số R^2 tăng lên 1 khi có thêm nhiều tính năng được đưa vào. Giữa: Quá trình huấn luyện Sai số bình phương trung bình (MSE) giảm xuống 0 khi số lượng đặc trưng được đưa vào càng nhiều. Bên phải: MSE của tập dữ liệu kiểm thử. Tăng lên khi có thêm nhiều tính năng được bổ sung.

Mỗi yếu tố đều hoàn toàn không liên quan đến câu trả lời. Như được thể hiện trong Trong hình, hệ số R^2 của mô hình tăng lên 1 khi số lượng tính năng được bao gồm trong... Mô hình tăng lên, và tương ứng, MSE của tập dữ liệu huấn luyện giảm xuống 0. khi số lượng tính năng tăng lên, ngay cả khi các tính năng đó hoàn toàn không liên quan đến phản hồi. Mặt khác, MSE trên một yếu tố độc lập Tập dữ liệu kiểm thử trở nên cực kỳ lớn khi số lượng tính năng được bao gồm trong đó tăng lên. Mô hình tăng lên vì việc bao gồm các biến dự báo bổ sung dẫn đến sự gia tăng đáng kể. sự gia tăng phương sai của các ước tính hệ số. Xem xét bài kiểm tra Với MSE được thiết lập, rõ ràng là mô hình tốt nhất chỉ chứa tối đa một vài biến. Tuy nhiên, người nào đó chỉ xem xét R^2 hoặc tập dữ liệu huấn luyện một cách hời hợt thì sẽ sai lầm. MSE có thể kết luận sai rằng mô hình có số lượng lớn nhất Việc xem xét tất cả các biến số là tốt nhất. Điều này cho thấy tầm quan trọng của việc hết sức cẩn thận. khi phân tích các tập dữ liệu có số lượng biến lớn, và luôn luôn Đánh giá hiệu suất mô hình trên một tập dữ liệu kiểm thử độc lập.



HÌNH 6.24. Thao tác lasso được thực hiện với $n = 100$ quan sát và ba các giá trị của p , số lượng đặc điểm. Trong số p đặc điểm, 20 đặc điểm được liên kết với phản hồi. Biểu đồ hộp hiển thị các giá trị MSE kiểm định thu được khi sử dụng ba phương pháp khác nhau. giá trị của tham số điều chỉnh λ trong (6.7). Để dễ hiểu hơn, thay vì Khi báo cáo λ , các bậc tự do được báo cáo; đối với phương pháp lasso thì điều này là đơn giản chỉ là số lượng các hệ số ước tính khác 0. Khi $p = 20$, thì Sai số bình phương trung bình (MSE) thấp nhất khi kiểm tra đạt được với mức độ điều chỉnh nhỏ nhất. Khi $p = 50$, giá trị MSE kiểm định thấp nhất đạt được khi có một lượng đáng kể. về phương pháp điều chỉnh. Khi $p = 2000$, thuật toán Lasso hoạt động kém hiệu quả bất kể mức độ điều chỉnh, do thực tế là chỉ có 20 trong số 2.000 tính năng thực sự có liên quan đến kết quả.

Trong Mục 6.1.3, chúng ta đã xem xét một số phương pháp điều chỉnh quá trình huấn luyện. Đặt RSS hoặc R^2 để tính đến số lượng biến được sử dụng để phù hợp.

một mô hình bình phương tối thiểu. Thật không may, các phương pháp C_p , AIC và BIC không phù hợp trong môi trường đa chiều, bởi vì việc ước tính σ^2

có vấn đề. (Ví dụ, công thức cho σ^2 từ Chương 3 cho ra một

(Ước tính $\sigma^2 = 0$ trong trường hợp này.) Tương tự, các vấn đề phát sinh trong ứng dụng.

của R^2 hiệu chỉnh trong thiết lập đa chiều, vì người ta có thể dễ dàng thu được

một mô hình với giá trị R^2 hiệu chỉnh là 1. Rõ ràng, các phương pháp tiếp cận thay thế

Cần có những giải pháp phù hợp hơn với môi trường đa chiều.

6.4.3 Hồi quy trong không gian đa chiều

Hóa ra nhiều phương pháp được trình bày trong chương này để điều chỉnh

các mô hình bình phương tối thiểu kém linh hoạt hơn, chẳng hạn như lựa chọn từng bước tiến lên, ridge

hồi quy, phương pháp lasso và hồi quy thành phần chính, đặc biệt là

Hữu ích cho việc thực hiện hồi quy trong môi trường đa chiều. Về cơ bản,

Các phương pháp này tránh hiện tượng quá khớp bằng cách sử dụng phương pháp khớp ít linh hoạt hơn.

so với phương pháp bình phương tối thiểu.

Hình 6.24 minh họa hiệu suất của thuật toán lasso trong một mô phỏng đơn giản.

Ví dụ. Có $p = 20, 50$ hoặc 2.000 đặc điểm, trong đó 20 đặc điểm thực sự đúng.

có liên quan đến kết quả. Thuật toán lasso được thực hiện trên 100 lượt huấn luyện ($n = 100$).

các quan sát, và sai số bình phương trung bình được đánh giá trên một nghiên cứu độc lập.

tập dữ liệu kiểm thử. Khi số lượng đặc trưng tăng lên, lỗi trên tập dữ liệu kiểm thử cũng tăng lên.

Khi $p = 20$, sai số tập dữ liệu xác thực thấp nhất đạt được khi λ trong

(6.7) nhỏ; tuy nhiên, khi p lớn hơn thì lỗi tập xác thực thấp nhất đạt được bằng cách sử dụng giá trị λ lớn hơn. Trong mỗi biểu đồ hộp, thay vì báo cáo các giá trị λ được sử dụng, bậc tự do của giải pháp lasso thu được được hiển thị; đây chỉ đơn giản là số lượng ước tính hệ số khác 0 trong giải pháp lasso, và là thước đo tính linh hoạt của phép khớp lasso. Hình 6.24 nêu bật ba điểm quan trọng: (1) điều chỉnh hoặc co rút đóng vai trò quan trọng trong các vấn đề có chiều cao, (2) lựa chọn tham số điều chỉnh phù hợp là rất quan trọng để có hiệu suất dự đoán tốt, và (3) lỗi kiểm tra có xu hướng tăng lên khi chiều của vấn đề (tức là số lượng đặc trưng hoặc biến dự đoán) tăng lên, trừ khi các đặc trưng bổ sung thực sự liên quan đến phản hồi.

Điểm thứ ba nêu trên thực chất là một nguyên tắc quan trọng trong phân tích dữ liệu đa chiều, được gọi là "lời nguyên của chiều không gian". Người ta có thể nghĩ rằng khi số lượng đặc trưng được sử dụng để xây dựng mô hình tăng lên, chất lượng của mô hình được xây dựng cũng sẽ tăng lên. Tuy nhiên, khi so sánh hai bảng bên trái và bên phải trong Hình 6.24, chúng ta thấy rằng điều này không nhất thiết đúng: trong ví dụ này, MSE trên tập dữ liệu kiểm tra gần như tăng gấp đôi khi p tăng từ 20 lên 2.000. Nói chung, việc thêm các đặc trưng tín hiệu bổ sung thực sự liên quan đến biến phản hồi sẽ cải thiện mô hình được xây dựng, theo nghĩa là dẫn đến giảm lỗi trên tập dữ liệu kiểm tra. Tuy nhiên, việc thêm các đặc trưng nhiễu không thực sự liên quan đến biến phản hồi sẽ dẫn đến sự suy giảm chất lượng của mô hình được xây dựng, và do đó làm tăng lỗi trên tập dữ liệu kiểm tra. Điều này là do các đặc trưng nhiễu làm tăng chiều không gian của bài toán, làm trầm trọng thêm nguy cơ quá khớp (vì các đặc trưng nhiễu có thể được gán các hệ số khác 0 do sự liên kết ngẫu nhiên với biến phản hồi trên tập huấn luyện) mà không mang lại bất kỳ lợi ích tiềm năng nào về mặt cải thiện lỗi trên tập dữ liệu kiểm tra. Như vậy, chúng ta thấy rằng các công nghệ mới cho phép thu thập các phép đo cho hàng nghìn hoặc hàng triệu đặc điểm là con dao hai lưỡi: chúng có thể dẫn đến các mô hình dự đoán được cải thiện nếu các đặc điểm này thực sự liên quan đến vấn đề đang xét, nhưng sẽ dẫn đến kết quả kém hơn nếu các đặc điểm đó không liên quan. Ngay cả khi chúng có liên quan, phương sai phát sinh trong việc hiệu chỉnh các hệ số của chúng có thể lớn hơn mức giảm sai lệch mà chúng mang lại.

lời nguyên
của chiều không gian

6.4.4 Giải thích kết quả trong không gian đa chiều

Khi thực hiện hồi quy Lasso, hồi quy Ridge, hoặc các thủ tục hồi quy khác trong môi trường đa chiều, chúng ta phải hết sức thận trọng trong cách báo cáo kết quả thu được. Trong Chương 3, chúng ta đã tìm hiểu về đa cộng tuyến, khái niệm cho rằng các biến trong hồi quy có thể tương quan với nhau. Trong môi trường đa chiều, vấn đề đa cộng tuyến trở nên nghiêm trọng hơn: bất kỳ biến nào trong mô hình đều có thể được viết dưới dạng tổ hợp tuyến tính của tất cả các biến khác trong mô hình. Về cơ bản, điều này có nghĩa là chúng ta không bao giờ có thể biết chính xác biến nào (nếu có) thực sự có khả năng dự đoán kết quả, và chúng ta không bao giờ có thể xác định được các hệ số tốt nhất để sử dụng trong hồi quy. Tối đa, chúng ta chỉ có thể hy vọng gán các hệ số hồi quy lớn cho các biến có tương quan với các biến thực sự có khả năng dự đoán kết quả.

6.5 Thực hành: Mô hình tuyến tính và phương pháp điều chỉnh 267

Ví dụ, giả sử chúng ta đang cố gắng dự đoán huyết áp dựa trên nửa triệu SNP, và phương pháp chọn lọc từng bước tiến lên cho thấy 17 trong số các SNP đó tạo ra một mô hình dự đoán tốt trên dữ liệu huấn luyện.

Sẽ là không chính xác nếu kết luận rằng 17 SNP này dự đoán huyết áp hiệu quả hơn các SNP khác không được đưa vào mô hình. Có thể có nhiều tập hợp 17 SNP khác cũng có thể dự đoán huyết áp tốt như mô hình đã chọn. Nếu chúng ta thu được một tập dữ liệu độc lập và thực hiện lựa chọn từng bước tiến lên trên tập dữ liệu đó, chúng ta có thể thu được một mô hình chứa một tập hợp SNP khác, và thậm chí có thể không trùng lặp. Điều này không làm giảm giá trị của mô hình thu được – ví dụ, mô hình có thể rất hiệu quả trong việc dự đoán huyết áp trên một nhóm bệnh nhân độc lập và có thể hữu ích về mặt lâm sàng cho các bác sĩ. Nhưng chúng ta phải cẩn thận không phóng đại kết quả thu được và phải làm rõ rằng những gì chúng ta đã xác định chỉ là một trong nhiều mô hình khả thi để dự đoán huyết áp, và nó cần được xác thực thêm trên các tập dữ liệu độc lập.

Điều quan trọng là phải đặc biệt cẩn thận khi báo cáo lỗi và các thước đo độ phù hợp của mô hình trong môi trường đa chiều. Chúng ta đã thấy rằng khi $p > n$, rất dễ thu được một mô hình vô dụng có phần dư bằng không. Do đó, không bao giờ nên sử dụng tổng bình phương sai số, giá trị p , thống kê R^2 hoặc các thước đo truyền thống khác về độ phù hợp của mô hình trên dữ liệu huấn luyện làm bằng chứng cho thấy mô hình phù hợp tốt trong môi trường đa chiều. Ví dụ, như chúng ta đã thấy trong Hình 6.23, người ta có thể dễ dàng thu được một mô hình với $R^2 = 1$ khi $p > n$.

Việc báo cáo thông tin này có thể khiến người khác hiểu nhầm rằng đã thu được một mô hình có giá trị thống kê và hữu ích, trong khi thực tế điều này hoàn toàn không cung cấp bằng chứng nào về một mô hình thuyết phục. Điều quan trọng là thay vào đó phải báo cáo kết quả trên một tập dữ liệu kiểm thử độc lập, hoặc các lỗi kiểm định chéo. Ví dụ, MSE hoặc R^2 trên một tập dữ liệu kiểm thử độc lập là một thước đo hợp lệ về độ phù hợp của mô hình, nhưng MSE trên tập dữ liệu huấn luyện chắc chắn không phải.

6.5 Bài thực hành: Mô hình tuyến tính và các phương pháp điều chỉnh

Trong phòng thí nghiệm này, chúng ta sẽ áp dụng nhiều kỹ thuật đã được thảo luận trong chương này. Chúng tôi nhập một số thư viện của mình ở cấp độ cao nhất này.

```
Trong [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots from
statsmodels.api import OLS import
sklearn.model_selection as skm import
sklearn.linear_model as skl from
sklearn.preprocessing import StandardScaler from ISLP
import load_data from
ISLP.models import ModelSpec as MS from
functools import partial
```

Chúng tôi lại tiếp tục thu thập các vật tư nhập khẩu cần thiết cho phòng thí nghiệm này.

```
Trong [2]: từ sklearn.pipeline import Pipeline từ
sklearn.decomposition import PCA
```

```
from sklearn.cross_decomposition import PLSRegression from ISLP.models import
\ (Stepwise, sklearn_selected,

sklearn_selection_path) !

pip install l0bnb from l0bnb
import fit_path
```

Chúng tôi đã cài đặt gói `l0bnb` ngay lập tức. Lưu ý lệnh `!pip install` được mã hóa – lệnh này được chạy như một lệnh hệ thống riêng biệt.

6.5.1 Phương pháp lựa chọn tập con

Ở đây, chúng tôi triển khai các phương pháp giảm số lượng tham số trong mô hình bằng cách giới hạn mô hình chỉ sử dụng một tập hợp con các biến đầu vào.

Lựa chọn tiến lên

Chúng tôi sẽ áp dụng phương pháp lựa chọn tiến lên (forward-selection) cho dữ liệu về các cầu thủ đánh bóng. Chúng tôi muốn dự đoán mức lương của một cầu thủ bóng chày dựa trên các số liệu thống kê khác nhau liên quan đến thành tích trong năm trước.

Trước hết, chúng ta lưu ý rằng biến `Salary` bị thiếu đối với một số cầu thủ. Hàm `np.isnan()` có thể được sử dụng để xác định các quan sát bị thiếu. Nó trả về một mảng có cùng kích thước với vector đầu vào, với `True` cho bất kỳ phần tử nào bị thiếu và `False` cho các phần tử không bị thiếu. Sau đó, phương thức `sum()` có thể được sử dụng để đếm tất cả các phần tử bị thiếu.

tổng()

```
Trong [3]: Hitters = load_data('Hitters')
np.isnan(Hitters['Salary']).sum()
```

Ra ngoài[3]: 59

Chúng ta thấy rằng thông tin về Lương bị thiếu đối với 59 cầu thủ. Phương thức `dropna()` của data frame loại bỏ tất cả các hàng có giá trị bị thiếu trong bất kỳ biến nào (theo mặc định – xem `Hitters.dropna()`).

```
Trong [4]: Hitters = Hitters.dropna(); Hitters.shape
```

Ra[4]: (263, 20)

Đầu tiên, chúng tôi chọn mô hình tốt nhất bằng cách sử dụng phương pháp lựa chọn tiến lên dựa trên Cp (6.2). Điểm số này không được tích hợp sẵn như một chỉ số trong `sklearn`. Do đó, chúng tôi định nghĩa một hàm để tự tính toán nó và sử dụng nó như một thước đo điểm số. Theo mặc định, `sklearn` cố gắng tối đa hóa điểm số, do đó hàm tính điểm của chúng tôi tính toán thống kê Cp âm.

```
Trong [5]: def nCp(sigma2, estimator, X, Y):
    "Thống kê Cp âm" n, p = X.shape
    Yhat =
    estimator.predict(X)
    RSS = np.sum((Y - Yhat)**2) return -(RSS
    + 2 * p * sigma2) / n
```


6.5 Thực hành: Mô hình tuyến tính và phương pháp điều chỉnh 269

Chúng ta cần ước tính phương sai dư σ^2 , đây là đối số đầu tiên trong hàm tính điểm ở trên. Chúng ta sẽ xây dựng mô hình lớn nhất, sử dụng tất cả các biến, và ước tính σ^2 dựa trên MSE của nó.

```
Trong [6]: design = MS(Hitters.columns.drop('Salary')).fit(Hitters)
Y = np.array(Hitters['Salary'])
X = design.transform(Hitters) sigma2 =
OLS(Y,X).fit().scale
```

Hàm `sklearn_selected()` mong đợi một bộ tính điểm chỉ có ba đối số – ba đối số cuối cùng trong định nghĩa của `nCp()` ở trên. Chúng ta sử dụng hàm `partial()` được thấy lần đầu trong Phần 5.3.3 để cố định đối số đầu tiên với ước tính σ^2 của chúng ta.

```
Trong [7]: neg_Cp = partial(nCp, sigma2)
```

Giờ đây chúng ta có thể sử dụng `neg_Cp()` làm công cụ chấm điểm để lựa chọn mô hình.

Cùng với điểm số, chúng ta cần xác định chiến lược tìm kiếm. Điều này được thực hiện thông qua đối tượng `Stepwise()` trong gói `ISLP.models`. Phương thức `Stepwise.first_peak()` chạy từng bước tiến lên cho đến khi bất kỳ sự bổ sung nào nữa vào mô hình không dẫn đến sự cải thiện điểm số đánh giá. Tương tự, phương thức `Stepwise.fixed_steps()` thực hiện một số bước tìm kiếm từng bước cố định.

```
Trong [8]: chiến lược = Stepwise.first_peak(thiết kế,
                                         hướng='tiến',
                                         max_terms=len(design.terms))
```

Giờ đây, chúng ta sẽ xây dựng một mô hình hồi quy tuyến tính với biến kết quả là **Lương**, sử dụng phương pháp chọn lọc tiến lên. Để làm điều đó, chúng ta sử dụng hàm `sklearn_selected()` từ gói `ISLP.models`. Hàm này nhận một mô hình từ `statsmodels` cùng với một chiến lược tìm kiếm và chọn một mô hình bằng phương pháp **phù hợp** của nó. Nếu không chỉ định đối số **tính điểm**, điểm số mặc định là MSE, và do đó tất cả 19 biến sẽ được chọn (kết quả đầu ra không được hiển thị).

`sklearn_selected()`

```
Trong [9]: hitters_MSE = sklearn_selected(OLS,
                                         chiến lược)
hitters_MSE.fit(Hitters, Y)
hitters_MSE.selected_state_
```

Việc sử dụng `neg_Cp` dẫn đến một mô hình nhỏ hơn, như dự kiến, chỉ với 10 biến. các đối tượng được chọn.

```
Trong [10]: hitters_Cp = sklearn_selected(OLS, strategy,
                                         scoring=neg_Cp)
hitters_Cp.fit(Hitters, Y)
hitters_Cp.selected_state_
```

```
Ra[10]: ('Hỗ trợ',
        'AtBat',
        'Mero Dơi',
        'CRBI',
        'CRuns',
        'CWalks',
        'Phân công',
```

```
'Không sai hết ăn khách',  
'PutOuts',  
(Đi bộ)
```

Lựa chọn giữa các mô hình bằng cách sử dụng phương pháp tập dữ liệu kiểm định và Kiểm định chéo

Thay vì sử dụng Cp, chúng ta có thể thử phương pháp kiểm định chéo để chọn mô hình trong quá trình chọn lọc tiến lên. Để làm được điều này, chúng ta cần một phương pháp lưu trữ toàn bộ đường dẫn các mô hình được tìm thấy trong quá trình chọn lọc tiến lên và cho phép dự đoán cho từng mô hình. Điều này có thể được thực hiện bằng trình ước lượng `sklearn_selection_path()` từ thư viện `ISLP.models`. Hàm `cross_val_predict()` từ `ISLP.models` tính toán các dự đoán được kiểm định chéo cho mỗi mô hình dọc theo đường dẫn, mà chúng ta có thể sử dụng để đánh giá MSE được kiểm định chéo dọc theo đường dẫn.

Ở đây, chúng ta định nghĩa một chiến lược phù hợp với toàn bộ đường dẫn lựa chọn tiến lên. Mặc dù có nhiều lựa chọn tham số cho `sklearn_selection_path()`, chúng ta sử dụng các giá trị mặc định ở đây, chọn mô hình ở mỗi bước dựa trên mức giảm RSS lớn nhất.

```
sklearn_selection_path(  
    cross_val_  
    predict())
```

```
Trong [11]: chiến_lược = Stepwise.fixed_steps(thiết_kế,  
                                             len(design.terms),  
                                             direction='forward')  
full_path = sklearn_selection_path(OLS, strategy)
```

Chúng tôi hiện đã áp dụng toàn bộ đường dẫn lựa chọn tiến lên cho dữ liệu `Hitters` và so sánh...
Tính toán các giá trị phù hợp.

```
Trong [12]: full_path.fit(Hitters, Y)  
Yhat_in = full_path.predict(Hitters)  
Yhat_in.shape
```

Ra ngoài[12]: (263, 20)

Điều này cung cấp cho chúng ta một mảng các giá trị đã được hiệu chỉnh – tổng cộng 20 bước, bao gồm cả giá trị trung bình đã hiệu chỉnh cho mô hình null – mà chúng ta có thể sử dụng để đánh giá MSE trong mẫu. Như dự đoán, MSE trong mẫu được cải thiện sau mỗi bước chúng ta thực hiện, cho thấy chúng ta phải sử dụng phương pháp xác thực hoặc kiểm định chéo để chọn số bước. Chúng ta cố định trục y trong khoảng từ 50.000 đến 250.000 để so sánh với MSE của tập dữ liệu kiểm định chéo và xác thực bên dưới, cũng như các phương pháp khác như hồi quy Ridge, Lasso và hồi quy thành phần chính.

```
Trong [13]: mse_fig, ax = subplots(figsize=(8,8))  
insample_mse = ((Yhat_in - Y[:,None])**2).mean(0) n_steps =  
insample_mse.shape[0]  
ax.plot(np.arange(n_steps),  
        insample_mse, 'k',  
        # màu đen nhãn='Trong mẫu')  
  
ax.set_ylabel('MSE',  
              fontsize=20)  
ax.set_xlabel('# bước tiến từng bước',  
              cỡ_chữ=20)  
ax.set_xticks(np.arange(n_steps)[::2]) ax.legend()
```

```
ax.set_ylim([50000,250000]);
```

Hãy chú ý đến biểu thức `None` trong `Y[:,None]` ở trên. Biểu thức này thêm một trục (chiều) vào mảng một chiều `Y`, cho phép nó được tái sử dụng khi trừ đi từ mảng hai chiều `Yhat_in`.

Giờ đây, chúng ta đã sẵn sàng sử dụng phương pháp kiểm định chéo để ước tính sai số kiểm thử dọc theo đường dẫn của mô hình. Chúng ta chỉ được sử dụng các quan sát huấn luyện để thực hiện tất cả các khía cạnh của việc hiệu chỉnh mô hình – bao gồm cả việc lựa chọn biến. Do đó, việc xác định mô hình nào có kích thước nhất định là tốt nhất phải được thực hiện chỉ bằng cách sử dụng các quan sát huấn luyện trong mỗi fold huấn luyện. Điểm này tuy nhỏ nhưng rất quan trọng. Nếu toàn bộ tập dữ liệu được sử dụng để chọn tập con tốt nhất ở mỗi bước, thì các sai số trên tập kiểm định và sai số kiểm định chéo mà chúng ta thu được sẽ không phải là ước tính chính xác về sai số kiểm thử.

Chúng tôi hiện tính toán các giá trị dự đoán được kiểm định chéo bằng phương pháp kiểm định chéo 5 lần.

```
Trong [14]: K=5
kfold = skm.KFold(K,
                  random_state=0,
                  shuffle=True)
Yhat_cv = skm.cross_val_predict(full_path,
                                Người đánh bóng,
                                Y, cv=kfold)
Yhat_cv.shape
```

`skm.KFold()`
`Ra ngoài[14]: (263, 20)`
`skm.cross_val_predict()`

Ma trận dự đoán `Yhat_cv` có cùng hình dạng với `Yhat_in`; điểm khác biệt là các dự đoán trong mỗi hàng, tương ứng với một chỉ số mẫu cụ thể, được thực hiện từ các mô hình được huấn luyện trên một tập dữ liệu huấn luyện không bao gồm hàng đó.

Tại mỗi mô hình dọc theo đường dẫn, chúng ta tính toán MSE trong mỗi fold kiểm định chéo. Chúng ta sẽ lấy trung bình các giá trị này để có được MSE trung bình, và cũng có thể sử dụng các giá trị riêng lẻ để tính toán ước lượng sơ bộ về sai số chuẩn của giá trị trung bình.⁹ Do đó, chúng ta phải biết các chỉ số kiểm tra cho mỗi lần chia kiểm định chéo. Điều này có thể được tìm thấy bằng cách sử dụng phương thức `split()` của `kfold`. Bởi vì chúng ta đã cố định trạng thái ngẫu nhiên ở trên, bất cứ khi nào chúng ta chia bất kỳ mảng nào có cùng số hàng với `Y`, chúng ta sẽ thu được cùng các chỉ số huấn luyện và kiểm tra, mặc dù chúng ta chỉ đơn giản là bỏ qua các chỉ số huấn luyện bên dưới.

```
Trong [15]: cv_mse = []
for train_idx, test_idx in kfold.split(Y): errors =
    (Yhat_cv[test_idx] - Y[test_idx,None])**2 cv_mse.append(errors.mean(0))
# trung bình cột cv_mse = np.array(cv_mse).T cv_mse.shape
```

`Ra[15]: (20, 5)`

9. Ước tính này còn sơ bộ vì năm ước tính sai số dựa trên các tập dữ liệu huấn luyện chồng chéo nhau, do đó chúng không độc lập.

272 6. Lựa chọn và điều chỉnh mô hình tuyến tính

Giờ đây, chúng ta thêm các ước tính lỗi kiểm định chéo vào biểu đồ MSE của mình. Chúng ta bao gồm lỗi trung bình trên năm lần kiểm định và ước tính sai số chuẩn của giá trị trung bình.

```
Trong [16]: ax.errorbar(np.arange(n_steps),
                    cv_mse.mean(1),
                    cv_mse.std(1) / np.sqrt(K),
                    label='Cross-validated', c='r')
                    # màu đỏ

ax.set_ylim([50000,250000])
ax.legend()
mse_fig
```

Để lặp lại các bước trên bằng cách sử dụng phương pháp tập dữ liệu kiểm chứng, chúng ta chỉ cần thay đổi đối số `cv` thành một tập dữ liệu kiểm chứng: một phép chia ngẫu nhiên dữ liệu thành tập huấn luyện và tập kiểm tra. Chúng ta chọn kích thước tập kiểm tra là 20%, tương tự như kích thước của mỗi tập kiểm tra trong phương pháp kiểm chứng chéo 5 lần.

skm.Shuffle
Tách ra()

```
Trong [17]: validation = skm.ShuffleSplit(n_splits=1, test_size=0.2,
                                       random_state=0)

for train_idx, test_idx in validation.split(Y):
    full_path.fit(Hitters.iloc[train_idx],
                  Y[train_idx])

    Yhat_val = full_path.predict(Hitters.iloc[test_idx]) errors = (Yhat_val
    - Y[test_idx,None])**2 validation_mse = errors.mean(0)
```

Đối với trường hợp MSE trong mẫu, phương pháp tập dữ liệu xác thực không cung cấp sai số chuẩn.

```
Trong [18]: ax.plot(np.arange(n_steps),
                  validation_mse,
                  'b--', # màu xanh lam, nhãn đường đứt
                  đoạn='Xác thực')

ax.set_xticks(np.arange(n_steps)[::2])
ax.set_ylim([50000,250000])
ax.legend()
mse_fig
```

Lựa chọn tập con tốt nhất

Phương pháp chọn lọc từng bước tiến là một quy trình lựa chọn tham lam ; ở mỗi bước, nó bổ sung thêm một biến vào tập hợp hiện tại. Bây giờ chúng ta áp dụng phương pháp chọn tập con tốt nhất cho dữ liệu `Hitters` , phương pháp này tìm kiếm tập hợp các biến dự đoán tốt nhất cho mọi kích thước tập con.

Chúng ta sẽ sử dụng gói `l0bnb` để thực hiện việc lựa chọn tập con tốt nhất.

Thay vì giới hạn tập con có kích thước nhất định, gói phần mềm này tạo ra một chuỗi các giải pháp bằng cách sử dụng kích thước tập con như một hình phạt chứ không phải là một ràng buộc.

Mặc dù sự khác biệt khá nhỏ, nhưng nó chỉ thực sự rõ ràng khi chúng ta tiến hành kiểm chứng chéo.

```
Trong [19]: D = design.fit_transform(Hitters)
D = D.drop('intercept', axis=1)
X = np.asarray(D)
```

Ở đây chúng tôi đã loại bỏ cột đầu tiên tương ứng với hệ số chặn, là $10b_{nb}$

sẽ khớp với điểm giao nhau một cách riêng biệt. Chúng ta có thể tìm đường đi bằng cách sử dụng hàm `fit_path()` chức năng.

```
Trong [20]: path = fit_path(X,  
                             Y,  
                             max_nonzeros=X.shape[1])
```

Hàm `fit_path()` trả về một danh sách có các giá trị bao gồm đường dẫn đã được điều chỉnh. Các hệ số là B , hệ số chặn là B_0 , cũng như một vài thuộc tính khác có liên quan liên quan đến thuật toán tìm đường cụ thể được sử dụng. Những chi tiết như vậy nằm ngoài phạm vi bài viết này của cuốn sách này.

Trong [21]: đường dẫn[3]

```
Out[21]: {'B': mảng([0.,          , 3.254844, 0.,          , 0.,          , 0.,          ],  
                  [0.,          , 0.,          , 0.,          , 0.,          , 0.,          ],  
                  [0.,          , 0.677753, 0.,          , 0.,          , 0.,          , 0.,          ],  
                  [0.,          , 0.,          , 0.,          , 0.,          , 0.] ),  
         'B0': -38.98216739555494,  
         'lambda_0': 0.011416248027450194,  
         'M': 0.5829861733382011,  
         'Thời gian vượt quá': Sai}
```

Trong ví dụ trên, ta thấy rằng ở bước thứ tư trong lô trình, ta có

hai hệ số khác 0 trong 'B', tương ứng với giá trị 0,114 cho

Tham số phạt `lambda_0`. Chúng ta có thể đưa ra dự đoán bằng cách sử dụng chuỗi này.

các phép tính phù hợp trên tập dữ liệu xác thực như một hàm của `lambda_0`, hoặc với nhiều công việc hơn sử dụng Kiểm định chéo.

6.5.2 Hồi quy Ridge và Lasso

Chúng ta sẽ sử dụng gói `sklearn.linear_model` (trong đó chúng ta sử dụng `skl` làm tên gói).

(viết tắt bên dưới) để phù hợp với các mô hình tuyến tính được điều chỉnh bằng ridge và lasso trên Dữ liệu về người đánh bóng. Chúng ta bắt đầu với ma trận mô hình X (không có hệ số chặn). mà chúng ta đã tính toán trong phần trước về hồi quy tập con tốt nhất.

Hồi quy Ridge

Chúng ta sẽ sử dụng hàm `skl.ElasticNet()` để huấn luyện cả mô hình hồi quy Ridge và Lasso. Để huấn luyện một đường dẫn của các mô hình hồi quy Ridge, chúng ta sử dụng `skl.ElasticNet.path()`, hàm này sẽ có thể phù hợp với cả phương pháp hồi quy Ridge và Lasso, cũng như sự kết hợp lai; hồi quy Ridge tương ứng với `l1_ratio=0`. Nên chuẩn hóa các cột.

của X trong các ứng dụng này, nếu các biến được đo bằng các đơn vị khác nhau.

Vì `skl.ElasticNet()` không thực hiện chuẩn hóa, chúng ta phải tự xử lý việc đó.

chính chúng ta. Vì chúng ta chuẩn hóa trước để tìm ước lượng hệ số.

Trên thang đo góc, chúng ta phải bỏ chuẩn hóa các ước lượng hệ số.

tham số λ trong (6.5) và (6.7) được gọi là **alpha** trong **sklearn**. Để được

Tương tự như các phần còn lại của chương này, chúng ta sử dụng `lambda` thay vì `alpha`.
trong phần tiếp theo. 10

10 Tại thời điểm xuất bản, Đường gờ phù hợp với đường gờ trong khối mã [22] vẫn đề không cần thiết. Các thông báo cảnh báo hội tụ; chúng tôi hy vọng những thông báo này sẽ biến mất khi gói phần mềm này hoàn thiện hơn.

```
Trong [22]: Xs = X - X.mean(0)[None,:]
X_scale = X.std(0)
Xs = Xs / X_scale[None,:]
lambdas = 10*np.linspace(8, -2, 100) / Y.std()
soln_array = skl.ElasticNet.path(Xs,
                                Y,
                                l1_ratio=0.,
                                alpha = lambda)[1]

soln_array.shape
```

Ra[22]: (19, 100)

Ở đây, chúng ta trích xuất mảng các hệ số tương ứng với các nghiệm.
dọc theo đường dẫn điều chỉnh. Theo mặc định, phương thức `skl.ElasticNet.path`
Nó khớp với một đường dẫn dọc theo phạm vi giá trị λ được chọn tự động, ngoại trừ
trường hợp khi `l1_ratio=0`, dẫn đến hồi quy Ridge (như trường hợp này)
ở đây). Vì vậy, ở đây chúng tôi đã chọn triển khai hàm trên một lưới của
các giá trị nằm trong khoảng từ $\lambda = 108$ đến $\lambda = 10^{-2}$ được chia tỷ lệ theo độ lệch chuẩn
của y , về cơ bản bao trùm toàn bộ các kịch bản từ mô hình null.
Chỉ bao gồm hệ số chặn, theo phương pháp bình phương tối thiểu.
Mỗi giá trị của λ tương ứng với một vectơ các hệ số hồi quy Ridge.
có thể truy cập được thông qua một cột của `soln_array`. Trong trường hợp này, `soln_array` là
Một ma trận 19×100 , với 19 hàng (mỗi hàng tương ứng với một biến dự đoán) và 100 cột.
(một giá trị tương ứng với mỗi giá trị của λ).
Chúng ta chuyển vị ma trận này và biến nó thành một khung dữ liệu để thuận tiện hơn.
xem và lập kế hoạch.

```
Trong [23]: soln_path = pd.DataFrame(soln_array.T,
                                   cột = D.cột,
                                   index=-np.log(lambdas))
soln_path.index.name = 'negative log(lambda)'
đường dẫn giải
```

Ra ngoài[23]:

	AtBat	Luật tay trái	HmRun	Chạy...
tiêu cực				
log(lambda)				
-12.310855	0.000800	0.000889	0.000695	0.000851 ...
-12.078271	0.001010	0.001122	0.000878	0.001074 ...
-11.845686	0.001274	0.001416	0.001107	0.001355 ...
-11.613102	0.001608	0.001787	0.001397	0.001710 ...
-11.380518	0.002029	0.002255	0.001763	0.002158 ...
...	...	...	...	...

100 hàng x 19 cột

Chúng ta vẽ đồ thị các đường đi để hiểu rõ hơn về sự biến đổi của các hệ số theo λ . Để
Để kiểm soát vị trí của chú thích, trước tiên chúng ta đặt thuộc tính `'legend'` thành `'False'` trong biểu đồ.
phương pháp, thêm nó sau đó bằng phương thức `legend()` của `ax`.

```
Trong [24]: path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
```

111) do khá mang tính kỹ thuật; đối với tất cả các mô hình ngoại trừ mô hình sườn núi, chúng ta đều có thể tìm thấy giá trị nhỏ nhất.
Giá trị của λ mà tại đó tất cả các hệ số đều bằng 0. Đối với đường gờ, giá trị này là ∞ .

```
ax.set_ylabel(' Hệ số chuẩn hóa', fontsize=20)
ax.legend(loc='upper left');
```

(Chúng tôi đã sử dụng định dạng **LaTeX** trong nhãn ngang để định dạng)
(ký hiệu Hy Lạp λ thích hợp.) Chúng tôi kỳ vọng các ước tính hệ số sẽ chính xác hơn nhiều.
nhỏ hơn, xét về chuẩn 2, khi sử dụng giá trị λ lớn, so với
khi sử dụng giá trị nhỏ của λ . (Hãy nhớ rằng chuẩn 2 là bình phương)
(căn bậc hai của tổng bình phương các giá trị hệ số.) Chúng tôi hiển thị các hệ số.
ở bước thứ 40, trong đó λ là 25,535.

```
Trong [25]: beta_hat = soln_path.loc[soln_path.index[39]]
          lambdas[39], beta_hat
```

Ra[25]: (25.535,

AtBat	5.433750
Liệt troy cấp	6.223582
HmRun	4.585498
Chạy	5.880855
đạt điểm	6.195921
Đi bộ	6.277975
Năm	5.299767
...	...

Hãy tính toán 2. Chuẩn của các hệ số chuẩn hóa.

```
Trong [26]: np.linalg.norm(beta_hat)
```

Ra[26]: 24.17

Ngược lại, đây là chuẩn 2 khi λ là $2,44e-01$. Lưu ý rằng nó lớn hơn nhiều.
2 chuẩn của các hệ số liên quan đến giá trị λ nhỏ hơn này.

```
Trong [27]: beta_hat = soln_path.loc[soln_path.index[59]]
          lambdas[59], np.linalg.norm(beta_hat)
```

Out[27]: (0.2437, 160.4237)

Ở trên, chúng ta đã chuẩn hóa X trước và sử dụng X để hiệu chỉnh mô hình Ridge .
Đối tượng `Pipeline()` trong `sklearn` cung cấp một cách rõ ràng để tách biệt quá trình
chuẩn hóa đặc trưng khỏi quá trình huấn luyện mô hình Ridge.

```
Trong [28]: ridge = skl.ElasticNet(alpha=lambdas[59], l1_ratio=0)
          scaler = StandardScaler(with_mean=True, with_std=True)
          pipe = Pipeline(steps=[('scaler', scaler), ('ridge', ridge)])
          pipe.fit(X, Y)
```

Chúng tôi chứng minh rằng nó cho cùng chuẩn 2 như trong phép hiệu chỉnh trước đây của chúng tôi trên
dữ liệu được chuẩn hóa.

```
Trong [29]: np.linalg.norm(ridge.coef_)
```

Ra[29]: 160.4237

Lưu ý rằng thao tác `pipe.fit(X, Y)` ở trên đã thay đổi đường gờ
đối tượng, và đặc biệt là đã thêm các thuộc tính như `'coef_'` mà trước đây không có.
Trước đó đã có mặt ở đó.

Ước tính sai số kiểm định của hồi quy Ridge

Việc chọn giá trị λ tiên nghiệm cho hồi quy Ridge rất khó, nếu không muốn nói là không thể. Chúng ta sẽ muốn sử dụng phương pháp kiểm định hoặc kiểm định chéo để chọn tham số điều chỉnh. Người đọc có thể không ngạc nhiên khi biết rằng phương pháp `Pipeline()` có thể được sử dụng trong `skm.cross_validate()` với cả phương pháp kiểm định (tức là `validation`) hoặc kiểm định chéo k-fold.

Chúng tôi cố định trạng thái ngẫu nhiên của bộ chia để kết quả thu được sẽ có thể tái tạo được.

```
Trong [30]: validation = skm.ShuffleSplit(n_splits=1,
                                         test_size=0.5,
                                         random_state=0)

ridge.alpha = 0.01
results = skm.cross_validate(ridge,
                              X,
                              Y, scoring='neg_mean_squared_error',
                              cv=validation)

- results['test_score']
```

```
Out[30]: array([134214.0])
```

Sai số bình phương trung bình (MSE) của tập dữ liệu kiểm tra là $1,342 \times 10^5$. Lưu ý rằng nếu thay vào đó chúng ta chỉ đơn giản là sử dụng mô hình chỉ có hệ số chặn, chúng ta sẽ dự đoán mỗi quan sát kiểm tra bằng cách sử dụng giá trị trung bình của các quan sát huấn luyện. Chúng ta có thể nhận được kết quả tương tự bằng cách sử dụng mô hình hồi quy Ridge với giá trị λ rất lớn. Lưu ý rằng `1e10` có nghĩa là 10^{10} .

```
Trong [31]: ridge.alpha = 1e10
results = skm.cross_validate(ridge,
                              X,
                              Y, scoring='neg_mean_squared_error',
                              cv=validation)

- results['test_score']
```

```
Out[31]: array([231788.32])
```

Rõ ràng việc chọn $\lambda = 0,01$ là tùy ý, vì vậy chúng ta sẽ sử dụng phương pháp kiểm định chéo hoặc phương pháp tập dữ liệu kiểm định để chọn tham số điều chỉnh λ . Đối tượng `GridSearchCV()` cho phép tìm kiếm lưới toàn diện để chọn tham số đó.

Trước tiên, chúng ta sử dụng phương pháp tập dữ liệu kiểm định để chọn λ .

Lưới
SearchCV()

```
Trong [32]: param_grid = {'ridge__alpha': lambdas} grid =
skm.GridSearchCV(pipe, param_grid,
                  cv=validation,
                  scoring='neg_mean_squared_error')

grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

```
Out[32]: Pipeline(steps=[('scaler', StandardScaler()),
                          ('ridge', ElasticNet(alpha=0.005899, l1_ratio=0))])
```


6.5 Thực hành: Mô hình tuyến tính và phương pháp điều chỉnh 277

Ngoài ra, chúng ta cũng có thể sử dụng phương pháp kiểm chứng chéo 5 lần.

```
Trong [33]: grid = skm.GridSearchCV(pipe,
                                   param_grid,
                                   cv=kfold,
                                   scoring='neg_mean_squared_error')

grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

Hãy nhớ lại chúng ta đã thiết lập đối tượng `kfold` cho phương pháp kiểm định chéo 5 lần trên trang 271.

Bây giờ chúng ta vẽ biểu đồ MSE được kiểm định chéo theo hàm của $\log(\lambda)$, trong đó độ co rút giảm dần từ trái sang phải.

```
Trong [34]: ridge_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            -grid.cv_results_['mean_test_score'],
            yerr=grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20) ax.set_ylabel('MSE
được kiểm định chéo', fontsize=20);
```

Người ta có thể sử dụng phương pháp kiểm định chéo với nhiều chỉ số khác nhau để chọn tham số. Chỉ số mặc định cho `skl.ElasticNet()` là R^2 kiểm thử. Hãy so sánh R^2 với MSE để kiểm định chéo ở đây.

```
Trong [35]: Grid_r2 = skm.GridSearchCV(pipe,
                                       param_grid,
                                       cv=kfold)

grid_r2.fit(X, Y)
```

Cuối cùng, hãy vẽ biểu đồ kết quả cho hệ số R^2 được kiểm định chéo.

```
Trong [36]: r2_fig, ax = subplots(figsize=(8,8)) ax.errorbar(-
np.log(lambdas),
                grid_r2.cv_results_['mean_test_score'],
                yerr=grid_r2.cv_results_['std_test_score'] / np.sqrt(K)
                )
ax.set_xlabel('$-\log(\lambda)$', fontsize=20); ax.set_ylabel('Giá
trị  $R^2$  được kiểm định chéo', fontsize=20);
```

Kiểm tra chéo nhanh cho các đường dẫn giải pháp

Các đường gờ, lasso và lưới đàn hồi có thể được điều chỉnh hiệu quả dọc theo một chuỗi các giá trị λ , tạo ra cái gọi là đường dẫn giải pháp hoặc đường dẫn điều chỉnh.

Do đó, có mã chuyên dụng để phù hợp với các đường dẫn như vậy và để chọn giá trị λ thích hợp bằng cách sử dụng phương pháp kiểm định chéo. Ngay cả với các phép chia giống hệt nhau, kết quả cũng sẽ không hoàn toàn trùng khớp với `lưới` của chúng ta ở trên vì việc chuẩn hóa từng đặc trưng trong `lưới` được thực hiện trên mỗi fold, trong khi ở `pipeCV` bên dưới, nó chỉ được thực hiện một lần. Tuy nhiên, kết quả vẫn tương tự vì quá trình chuẩn hóa tương đối ổn định giữa các fold.

```
Trong [37]: RidgeCV = skl.ElasticNetCV(alphas=lambdas, l1_ratio=0,
                                       cv=kfold)

pipeCV = Pipeline(steps=[('scaler', scaler),
```

278 6. Lựa chọn và điều chỉnh mô hình tuyến tính

```
pipeCV.fit(X, Y)

('ridge', ridgeCV))
```

Hãy vẽ lại biểu đồ lỗi kiểm định chéo để thấy rằng nó...
tương tự như sử dụng `skm.GridSearchCV`.

```
Trong [38]: tuned_ridge = pipeCV.named_steps['ridge'].ridgeCV_fig,
ax = subplots(figsize=(8,8)) ax.errorbar(-np.log(lambdas),

tuned_ridge.mse_path_.mean(1),
yerr=tuned_ridge.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_ridge.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20) ax.set_ylabel('MSE
được kiểm định chéo', fontsize=20);
```

Ta thấy giá trị của λ dẫn đến sai số kiểm định chéo nhỏ nhất là $1,19e-02$, có sẵn dưới dạng giá trị `tuned_ridge.alpha_`. Vậy MSE kiểm định tương ứng với giá trị λ này là bao nhiêu?

```
Trong [39]: np.min(tuned_ridge.mse_path_.mean(1))
```

```
Ra[39]: 115526.71
```

Điều này thể hiện sự cải thiện hơn nữa so với MSE kiểm tra mà chúng ta thu được khi sử dụng $\lambda = 4$. Cuối cùng, `tuned_ridge.coef_` chứa các hệ số được hiệu chỉnh trên toàn bộ tập dữ liệu ở giá trị λ này.

```
Trong [40]: tuned_ridge.coef_
```

```
Out[40]: array([-222.80877051, 238.77246614, 3.21103754, -2.93050845, 3.64888723, 108.90953869,
-50.81896152, -105.15731984, , 210.35170348, 118.05683748, -150.21959435,
122.00714801, 57.1859509 30.36634231, -61.62459095, 77.73832472,
40.07350744, -25.02151514, -13.68429544])
```

Đúng như dự đoán, không có hệ số nào bằng 0 – hồi quy Ridge không thực hiện chọn lọc biến!

Đánh giá sai số kiểm thử của Ridge được kiểm định chéo

Việc chọn λ bằng phương pháp kiểm định chéo cung cấp một ước lượng hồi quy duy nhất, tương tự như việc khớp một mô hình hồi quy tuyến tính như chúng ta đã thấy trong Chương 3. Do đó, việc ước tính sai số kiểm định của nó là hợp lý. Tuy nhiên, chúng ta gặp phải vấn đề ở đây là kiểm định chéo sẽ sử dụng toàn bộ dữ liệu để chọn λ , do đó chúng ta không còn dữ liệu nào khác để ước tính sai số kiểm định. Một giải pháp thỏa hiệp là chia dữ liệu ban đầu thành hai tập hợp không giao nhau: một tập huấn luyện và một tập kiểm tra. Sau đó, chúng ta khớp một mô hình hồi quy Ridge được điều chỉnh bằng kiểm định chéo trên tập huấn luyện và đánh giá hiệu suất của nó trên tập kiểm tra. Chúng ta có thể gọi đây là phương pháp kiểm định chéo lồng ghép trong tập dữ liệu kiểm định. Về mặt lý thuyết, không có lý do gì để sử dụng một nửa dữ liệu cho mỗi trong hai tập dữ liệu để kiểm định.

Dưới đây, chúng ta sử dụng 75% dữ liệu cho huấn luyện và 25% cho kiểm tra, với mô hình ước lượng là hồi quy Ridge được tinh chỉnh bằng phương pháp kiểm định chéo 5 lần. Điều này có thể được thực hiện trong mã như sau:

```
Trong [41]: outer_valid = skm.ShuffleSplit(n_splits=1,
                                          test_size=0.25,
                                          random_state=1)

inner_cv = skm.KFold(n_splits=5,
                     shuffle=True,
                     random_state=2)

ridgeCV = skl.ElasticNetCV(alphas=lambdas,
                           l1_ratio=0,
                           cv=inner_cv)

pipeCV = Pipeline(steps=[('scaler', scaler),
                           ('ridge', ridgeCV)]);
```

```
Trong [42]: results = skm.cross_validate(pipeCV,
                                         X,
                                         Y, cv=outer_valid,
                                         scoring='neg_mean_squared_error')

-print(results['test_score'])
```

```
Out[42]: array([132393.84])
```

Dây thừng

Chúng ta đã thấy rằng hồi quy Ridge với lựa chọn λ khôn ngoan có thể vượt trội hơn phương pháp bình phương tối thiểu cũng như mô hình null trên tập dữ liệu `Hitters`. Bây giờ chúng ta đặt câu hỏi liệu lasso có thể tạo ra một mô hình chính xác hơn hoặc dễ hiểu hơn so với hồi quy Ridge hay không. Để phù hợp với mô hình lasso, chúng ta lại sử dụng hàm `ElasticNetCV()`; tuy nhiên, lần này chúng ta sử dụng đối số `l1_ratio=1`. Ngoài thay đổi đó, chúng ta tiến hành giống như khi phù hợp với mô hình Ridge.

```
Trong [43]: lassoCV = skl.ElasticNetCV(n_alphas=100, l1_ratio=1,
                                     cv=kfold)

pipeCV = Pipeline(steps=[('scaler', scaler),
                           ('lasso', lassoCV)])

pipeCV.fit(X, Y)
tuned_lasso = pipeCV.named_steps['lasso']
tuned_lasso.alpha_
```

```
Ra ngoài[43]: 3.147
```

```
Trong [44]: lambdas , soln_array = skl.Lasso.path(Xs,
                                                  Y, l1_ratio=1,
                                                  n_alphas=100)[:2]

soln_path = pd.DataFrame(soln_array.T,
                          columns=D.columns,
                          index=np.log(lambdas))
```

Từ biểu đồ hệ số của các hệ số chuẩn hóa, ta có thể thấy rằng tùy thuộc vào việc lựa chọn tham số điều chỉnh, một số hệ số sẽ bằng chính xác không.

```
Trong [45]: path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.legend(loc='upper left')
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Hệ số chuẩn hóa', fontsize=20);
```

Sai số kiểm định chéo nhỏ nhất thấp hơn MSE của tập dữ liệu kiểm tra của giả thuyết không. mô hình và phương pháp bình phương tối thiểu, và rất giống với MSE kiểm định là 115526,71. của hồi quy Ridge (trang 278) với λ được chọn bằng phương pháp kiểm định chéo.

```
Trong [46]: np.min(tuned_lasso.mse_path_.mean(1))
```

Ra ngoài[46]: 114690.73

Chúng ta hãy vẽ lại biểu đồ về lỗi kiểm định chéo.

```
Trong [47]: lassoCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(tuned_lasso.alphas_),
            tuned_lasso.mse_path_.mean(1),
            yerr=tuned_lasso.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_lasso.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Sai số bình phương trung bình được kiểm định chéo', fontsize=20);
```

Tuy nhiên, phương pháp lasso có ưu thế vượt trội so với phương pháp hồi quy Ridge trong... Các ước tính hệ số thu được rất thưa thớt. Ở đây chúng ta thấy rằng 6 trong số 19 ước lượng hệ số chính xác bằng không. Vì vậy, mô hình lasso với λ được chọn Phương pháp kiểm định chéo chỉ chứa 13 biến.

```
Trong [48]: tuned_lasso.coef_
```

```
Out[48]: array([-210.01008773, 243.4550306, ..., 0., 0.,
               0., 97.69397357, -41.52283116, -0., 39.62298193,
               0., 205.75273856, 124.55456561,
               -126.29986768, 15.70262427, -59.50157967, 75.24590036,
               21.62698014, -12.04423675, -0., ...])
```

Tương tự như hồi quy Ridge, chúng ta có thể đánh giá sai số kiểm định của phương pháp kiểm định chéo. Lasso bằng cách chia tập dữ liệu thành tập huấn luyện và tập kiểm thử trước, sau đó chạy thuật toán nội bộ. Kiểm định chéo trên tập dữ liệu huấn luyện. Chúng tôi để phần này lại như một bài tập.

6.5.3 Hồi quy PCR và PLS

Hồi quy thành phần chính

Phân tích hồi quy thành phần chính (PCR) có thể được thực hiện bằng cách sử dụng hàm `PCA()` từ mô-đun `sklearn.decomposition`. Bây giờ chúng ta sẽ áp dụng PCR cho nhóm `Hitters`. dữ liệu, để dự đoán mức lương. Một lần nữa, hãy đảm bảo rằng các giá trị bị thiếu được điền đầy đủ. đã bị xóa khỏi dữ liệu, như được mô tả trong Mục 6.5.1.

Chúng ta sử dụng hàm `LinearRegression()` để khớp mô hình hồi quy ở đây. Lưu ý rằng nó khớp một hệ số chặn theo mặc định, không giống như hàm `OLS()` đã thấy trước đó. Mục 6.5.1.

Tuyến tính
Hồi quy()

```
Trong [49]: pca = PCA(n_components=2)
linreg = skl.LinearRegression() pipe =
Pipeline([('pca', pca), ('linreg',
                        linreg)])
pipe.fit(X, Y)
pipe.named_steps['linreg'].coef_
```

```
Out[49]: array([0.09846131, 0.4758765 ])
```

Khi thực hiện PCA, kết quả sẽ khác nhau tùy thuộc vào việc dữ liệu đã được chuẩn hóa hay chưa. Như trong các ví dụ trước, điều này có thể được thực hiện bằng cách thêm một bước bổ sung vào quy trình.

```
Trong [50]: pipe = Pipeline([('scaler', scaler),
                            ('pca', pca),
                            ('linreg', linreg)])
pipe.fit(X, Y)
pipe.named_steps['linreg'].coef_
```

```
Out[50]: array([106.36859204, -21.60350456])
```

Đĩ nhiên, chúng ta có thể sử dụng CV để chọn số lượng thành phần bằng cách sử dụng `skm.GridSearchCV`, trong trường hợp này là cố định các tham số để thay đổi `n_components`.

```
Trong [51]: param_grid = {'pca__n_components': range(1, 20)} grid =
skm.GridSearchCV(pipe, param_grid,
                 cv=kfold,
                 scoring='neg_mean_squared_error')
lưới.phù hợp(X, Y)
```

Chúng ta hãy vẽ đồ thị kết quả như đã làm với các phương pháp khác.

```
Trong [52]: pcr_fig, ax = subplots(figsize=(8,8))
n_comp = param_grid['pca__n_components']
ax.errorbar(n_comp,
            -grid.cv_results_['mean_test_score'],
            grid.cv_results_['std_test_score'] / np.sqrt(K)) ax.set_ylabel('MSE
được kiểm định chéo', fontsize=20) ax.set_xlabel('# thành phần
chính', fontsize=20) ax.set_xticks(n_comp[::2]) ax.set_ylim([50000,250000]);
```

Chúng ta thấy rằng sai số kiểm định chéo nhỏ nhất xảy ra khi sử dụng 17 thành phần. Tuy nhiên, từ biểu đồ, chúng ta cũng thấy rằng sai số kiểm định chéo gần như tương tự khi chỉ có một thành phần được đưa vào mô hình. Điều này cho thấy rằng một mô hình chỉ sử dụng một số lượng nhỏ các thành phần có thể là đủ.

Điểm CV được cung cấp cho mỗi số lượng thành phần có thể có từ 1 đến 19 (bao gồm cả 1 và 19). Phương thức `PCA()` sẽ báo lỗi nếu chúng ta cố gắng chỉ điều chỉnh hệ số chặn với `n_components=0`, vì vậy chúng ta cũng tính toán MSE chỉ cho mô hình rỗng với các phép chia này.

```
Trong [53]: Xn = np.zeros((X.shape[0], 1))
cv_null = skm.cross_validate(linreg,
```

```

Xn,

Y, cv=kfold,
scoring='neg_mean_squared_error')

-cv_null['test_score'].mean()

```

Ra ngoài[53]: 204139.31

Thuộc tính `explained_variance_ratio_` của đối tượng `PCA` cung cấp tỷ lệ phần trăm phương sai được giải thích trong các biến dự đoán và trong biến phản hồi bằng cách sử dụng số lượng thành phần khác nhau. Khái niệm này được thảo luận chi tiết hơn trong Phần 12.2.

Trong [54]: `pipe.named_steps['pca'].explained_variance_ratio_`

Out[54]: array([0.3831424 , 0.21841076])

Nói một cách ngắn gọn, ta có thể hiểu đây là lượng thông tin về các biến dự báo được thu thập bằng cách sử dụng M thành phần chính. Ví dụ, khi $M = 1$, chỉ thu thập được 38,31% phương sai, trong khi $M = 2$ thu thập thêm 21,84%, tổng cộng là 60,15% phương sai. Đến $M = 6$, con số này tăng lên 88,63%. Sau đó, mức tăng tiếp tục giảm cho đến khi ta sử dụng tất cả $M = p = 19$ thành phần, thu thập được toàn bộ 100% phương sai.

Phương pháp bình phương tối thiểu từng phần

Phương pháp bình phương tối thiểu từng phần (PLS) được triển khai trong hàm `PLSRegression()`.

PLS

Trong [55]: `pls = PLSRegression(n_components=2, scale=True)`
`pls.fit(X, Y)`

Hồi quy()

Tương tự như trong PCR, chúng ta sẽ sử dụng CV để chọn số lượng thành phần.

Trong [56]: `param_grid = {'n_components':range(1, 20)}`
`lưới = skm.GridSearchCV(pls,`
`param_grid,`
`cv=kfold,`
`scoring='neg_mean_squared_error')`
`lưới.phù hợp(X, Y)`

Đối với các phương pháp khác, chúng tôi vẽ biểu đồ MSE.

Trong [57]: `pls_fig, ax = subplots(figsize=(8,8)) n_comp =`
`param_grid['n_components'] ax.errorbar(n_comp,`
`-grid.cv_results_['mean_test_score'],`
`grid.cv_results_['std_test_score'] / np.sqrt(K)) ax.set_ylabel('MSE`
`được kiểm định chéo', fontsize=20) ax.set_xlabel('# thành phần`
`chính', fontsize=20) ax.set_xticks(n_comp[:,2]) ax.set_ylim([50000,250000]);`

Sai số CV được giảm thiểu ở mức 12, mặc dù sự khác biệt không đáng kể.

Khoảng cách giữa điểm này và một con số thấp hơn nhiều, chẳng hạn như 2 hoặc 3 thành phần.

6.6 Bài tập

Khái niệm

1. Chúng ta thực hiện chọn tập con tốt nhất, chọn từng bước tiến và chọn từng bước lùi trên một tập dữ liệu duy nhất. Với mỗi phương pháp, chúng ta thu được $p + 1$ mô hình, chứa lần lượt 0, 1, 2, ..., p biến dự đoán. Hãy giải thích câu trả lời của bạn:

(a) Trong ba mô hình có k biến dự đoán, mô hình nào có RSS huấn luyện nhỏ nhất?

(b) Trong ba mô hình có k biến dự đoán, mô hình nào có giá trị nhỏ nhất?

Kiểm tra RSS?

(c) Đúng hay Sai:

- i. Các biến dự báo trong mô hình k biến được xác định bằng phương pháp chọn lọc từng bước tiến là một tập con của các biến dự báo trong mô hình $(k+1)$ biến được xác định bằng phương pháp chọn lọc từng bước tiến. ii. Các biến dự báo

trong mô hình k biến được xác định bằng phương pháp chọn lọc từng bước lùi là một tập con của các biến dự báo trong mô hình $(k + 1)$ biến được xác định bằng phương pháp chọn lọc từng bước lùi. iii. Các biến dự báo trong mô hình k biến được xác định

bằng phương pháp chọn lọc từng bước lùi là một tập con của các biến dự báo trong mô hình $(k + 1)$ biến được xác định bằng phương pháp chọn lọc từng bước tiến. iv. Các biến dự báo trong mô hình k biến được xác định bằng phương pháp chọn lọc từng

bước tiến là một tập con của các biến dự báo trong mô hình $(k+1)$ biến được xác định bằng phương pháp chọn lọc từng bước lùi. v. Các biến dự báo trong mô hình k biến được xác định bằng phương pháp chọn tập con tốt nhất là một tập con của

các biến dự báo trong mô hình $(k + 1)$ biến được xác định bằng phương pháp chọn tập con tốt nhất.

2. Đối với các phần (a) đến (c), hãy cho biết phần nào trong các phần i. đến iv. là đúng.

Hãy giải thích câu trả lời của bạn.

(a) So với phương pháp bình phương tối thiểu, phương pháp

lasso có những đặc điểm sau: i. Linh hoạt hơn và do đó sẽ cho độ chính xác dự đoán tốt hơn khi mức tăng độ lệch của nó nhỏ hơn mức giảm phương sai.

ii. Linh hoạt hơn và do đó sẽ cho độ chính xác dự đoán được cải thiện khi mức tăng phương sai của nó nhỏ hơn mức giảm độ lệch.

iii. Ít linh hoạt hơn và do đó sẽ cho độ chính xác dự đoán được cải thiện khi mức tăng độ lệch của nó nhỏ hơn mức giảm phương sai.

iv. Ít linh hoạt hơn và do đó sẽ cho độ chính xác dự đoán được cải thiện khi mức tăng phương sai của nó nhỏ hơn mức giảm độ lệch.

(b) Lập lại (a) đối với hồi quy Ridge so với phương pháp bình phương nhỏ nhất. (c)

Lập lại (a) đối với các phương pháp phi tuyến tính so với phương pháp bình phương nhỏ nhất.

3. Giả sử ta ước lượng các hệ số hồi quy trong mô hình hồi quy tuyến tính bằng cách tối thiểu hóa

$$\sum_{i=1}^N y_i - \beta_0 - \sum_{j=1}^P \beta_j x_{ij} \quad \text{tùy thuộc vào} \quad |\beta_j| \leq s$$

với một giá trị cụ thể của s . Đối với các phần (a) đến (e), hãy cho biết phần nào trong các phần i đến v là đúng. Giải thích câu trả lời của bạn.

- (a) Khi ta tăng s từ 0, RSS huấn luyện sẽ:
- i. Ban đầu tăng, sau đó dần dần bắt đầu giảm theo hình chữ U ngược.
 - ii. Ban đầu giảm, sau đó dần dần bắt đầu tăng theo hình chữ U ngược.
 - iii. Tăng dần.
 - iv. Giảm dần.
 - v. Không đổi.
- (b) Lặp lại (a) cho RSS kiểm định.
- (c) Lặp lại (a) cho phương sai.
- (d) Lặp lại (a) cho độ lệch (bình phương).
- (e) Lặp lại (a) cho sai số không thể giảm thiểu.

4. Giả sử ta ước lượng các hệ số hồi quy trong mô hình hồi quy tuyến tính bằng cách tối thiểu hóa

$$\sum_{i=1}^N y_i - \beta_0 - \sum_{j=1}^P \beta_j x_{ij} + \lambda \sum_{j=1}^P \beta_j^2$$

với một giá trị cụ thể của λ . Đối với các phần (a) đến (e), hãy chỉ ra câu nào trong các câu từ i đến v là đúng. Giải thích câu trả lời của bạn.

- (a) Khi ta tăng λ từ 0, RSS huấn luyện sẽ:
- i. Ban đầu tăng, sau đó dần dần bắt đầu giảm theo hình chữ U ngược.
 - ii. Ban đầu giảm, sau đó dần dần bắt đầu tăng theo hình chữ U ngược.
 - iii. Tăng dần đều.
 - iv. Giảm dần đều.
 - v. Giữ nguyên không đổi.
- (b) Lặp lại (a) cho RSS kiểm định.
- (c) Lặp lại (a) cho phương sai.
- (d) Lặp lại (a) cho độ lệch (bình phương).
- (e) Lặp lại (a) cho sai số không thể giảm thiểu.



5. Ai cũng biết rằng hồi quy Ridge thường cho các giá trị hệ số tương tự nhau đối với các biến có tương quan, trong khi hồi quy Lasso có thể cho các giá trị hệ số khá khác nhau đối với các biến có tương quan. Bây giờ chúng ta sẽ khám phá đặc tính này trong một thiết lập rất đơn giản.

Giả sử $n = 2$, $p = 2$, $x_{11} = x_{12}$, $x_{21} = x_{22}$. Hơn nữa, giả sử $y_1 + y_2 = 0$ và $x_{11} + x_{21} = 0$ và $x_{12} + x_{22} = 0$, sao cho ước lượng cho hệ số chặn trong mô hình hồi quy bình phương nhỏ nhất, hồi quy Ridge hoặc Lasso bằng 0: $\beta^0 = 0$.

(a) Hãy viết ra bài toán tối ưu hóa hồi quy Ridge trong tập hợp này.

(b) Hãy lập luận rằng trong bối cảnh này, các ước tính hệ số Ridge thỏa mãn $\beta^1 = \beta^2$.

(c) Viết ra bài toán tối ưu hóa lasso trong bối cảnh này. (d) Chứng

minh rằng trong bối cảnh này, các hệ số lasso β^1 và β^2 không phải là duy nhất –nói cách khác, có nhiều nghiệm khả thi cho bài toán tối ưu hóa trong (c). Mô tả các nghiệm này.

6. Bây giờ chúng ta sẽ tìm hiểu thêm về (6.12) và (6.13) .

(a) Xét (6.12) với $p = 1$. Với một số lựa chọn của y_1 và $\lambda > 0$, hãy vẽ đồ thị (6.12) theo hàm của β_1 . Đồ thị của bạn phải xác nhận rằng (6.12) được giải bởi (6.14). (b) Xét (6.13)

với $p = 1$. Với một số lựa chọn của y_1 và $\lambda > 0$, hãy vẽ đồ thị (6.13) theo hàm của β_1 . Đồ thị của bạn phải xác nhận rằng (6.13) được giải bởi (6.15).

7. Bây giờ chúng ta sẽ tìm ra mối liên hệ Bayes với thuật toán lasso và ridge.

Sự hồi quy được thảo luận trong Phần 6.2.2.



(a) Giả sử $y_i = \beta_0 + p \sum_{j=1}^n x_{ij}\beta_j + \epsilon_i$ trong đó $i = 1, \dots, n$ là độc lập phụ thuộc và được phân phối giống hệt nhau từ phân phối $N(0, \sigma^2)$. Hãy viết ra xác suất cho dữ liệu đó.

(b) Giả sử phân bố tiên nghiệm sau cho $\beta: \beta_1, \dots, \beta_p$ là các biến ngẫu nhiên độc lập và phân bố giống hệt nhau theo phân bố mũ kép với trung bình bằng 0 và tham số tỷ lệ chung là b : tức là $p(\beta) = \exp(-|\beta|/b)$. Hãy viết ra phân bố hậu nghiệm cho β trong trường hợp này.

(c) Hãy chứng minh rằng ước lượng lasso là giá trị mode của β theo phân bố hậu nghiệm này.

(d) Bây giờ giả sử phân bố tiên nghiệm sau cho $\beta: \beta_1, \dots, \beta_p$ là các biến ngẫu nhiên độc lập và phân bố giống hệt nhau theo phân phối chuẩn với trung bình bằng 0 và phương sai c . Hãy viết ra phân bố hậu nghiệm cho β trong trường hợp này.

(e) Hãy chứng minh rằng ước lượng hồi quy Ridge vừa là giá trị mode vừa là giá trị trung bình của β theo phân bố hậu nghiệm này.

Đã áp dụng

8. Trong bài tập này, chúng ta sẽ tạo ra dữ liệu mô phỏng, và sau đó sẽ sử dụng dữ liệu này để thực hiện lựa chọn từng bước tiến và lùi.

(a) Tạo một bộ tạo số ngẫu nhiên và sử dụng phương thức `normal()` của nó để tạo ra một vectơ dự đoán X có độ dài $n = 100$, cũng như một vectơ nhiễu có độ dài $n = 100$. (b) Tạo một vectơ phản

hồi Y có độ dài $n = 100$ theo mô hình

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon,$$

trong đó β_0 , β_1 , β_2 và β_3 là các hằng số do bạn chọn. (c) Sử dụng

phương pháp chọn từng bước tiến để chọn một mô hình chứa các biến dự đoán X_1 , X_2 , ..., X_{10} . Mô hình thu được theo C_p là gì? Báo cáo các hệ số của mô hình thu được. (d) Lặp lại (c), sử dụng phương pháp chọn từng bước lùi. Mô hình của bạn như thế nào?

câu trả lời so sánh với kết quả ở (c)?

(e) Bây giờ hãy áp dụng mô hình lasso cho dữ liệu mô phỏng, một lần nữa sử dụng X_1 , X_2 , ..., X_{10} làm các biến dự đoán. Sử dụng phương pháp kiểm định chéo để chọn giá trị tối ưu của λ . Tạo đồ thị biểu diễn sai số kiểm định chéo theo hàm của λ . Báo cáo các ước tính hệ số thu được và thảo luận về các kết quả đạt được.

(f) Bây giờ hãy tạo vectơ phản hồi Y theo mô hình.

$$Y = \beta_0 + \beta_7 X + \epsilon,$$

và thực hiện lựa chọn từng bước tiến và thuật toán lasso. Thảo luận về các kết quả thu được.

9. Trong bài tập này, chúng ta sẽ dự đoán số lượng đơn đăng ký nhận được. sử dụng các biến khác trong tập dữ liệu của trường đại học.

(a) Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra. (b) Áp dụng

mô hình tuyến tính bằng phương pháp bình phương tối thiểu trên tập huấn luyện và báo cáo sai số kiểm tra thu được. (c) Áp dụng

mô hình hồi quy Ridge trên tập huấn luyện, với λ được chọn bằng phương pháp kiểm định chéo. Báo cáo sai số kiểm tra thu được. (d) Áp dụng mô hình Lasso

trên tập huấn luyện, với λ được chọn bằng phương pháp kiểm định chéo. Báo cáo sai số kiểm tra thu được, cùng với số lượng ước lượng hệ số khác 0.

(e) Xây dựng mô hình PCR trên tập dữ liệu huấn luyện, với M được chọn bằng phương pháp kiểm định chéo. Báo cáo sai số kiểm định thu được, cùng với giá trị M được chọn bằng phương pháp kiểm định chéo.

(f) Xây dựng mô hình PLS trên tập dữ liệu huấn luyện, với M được chọn bằng phương pháp kiểm định chéo. Báo cáo sai số kiểm định thu được, cùng với giá trị M được chọn bằng phương pháp kiểm định chéo.

- (g) Nhận xét về kết quả thu được. Chúng ta có thể dự đoán số lượng đơn xin nhập học đại học chính xác đến mức nào? Có sự khác biệt đáng kể nào giữa các lỗi kiểm tra phát sinh từ năm phương pháp này không?

10. Chúng ta đã thấy rằng khi số lượng đặc trưng được sử dụng trong mô hình tăng lên, lỗi huấn luyện chắc chắn sẽ giảm, nhưng lỗi kiểm thử có thể không giảm.

Chúng ta sẽ cùng tìm hiểu điều này trong một bộ dữ liệu mô phỏng.

- (a) Tạo một tập dữ liệu với $p = 20$ đặc trưng, $n = 1.000$ quan sát và một vector phản hồi định lượng tương ứng được tạo ra theo mô hình.

$$Y = X\beta + \epsilon$$

trong đó β có một số phần tử bằng chính xác 0. (b) Chia tập dữ liệu

của bạn thành một tập huấn luyện chứa 100 quan sát và một tập dữ liệu thử nghiệm chứa 900 quan sát.

- (c) Thực hiện lựa chọn tập con tốt nhất trên tập huấn luyện và vẽ biểu đồ MSE của tập huấn luyện tương ứng với mô hình tốt nhất của mỗi kích thước. (d) Vẽ biểu đồ MSE của tập kiểm tra tương ứng với mô hình tốt nhất của mỗi kích thước kích cỡ.

- (e) Với kích thước mô hình nào thì MSE trên tập dữ liệu kiểm tra đạt giá trị nhỏ nhất? Hãy bình luận về kết quả của bạn. Nếu nó đạt giá trị nhỏ nhất với mô hình chỉ chứa hệ số chặn hoặc mô hình chứa tất cả các đặc trưng, thì hãy thử nghiệm với cách bạn tạo dữ liệu trong (a) cho đến khi bạn tìm ra kích thước mà MSE trên tập dữ liệu kiểm tra được tối thiểu hóa với kích thước mô hình trung gian.

- (f) Mô hình có MSE trên tập dữ liệu kiểm tra được tối thiểu hóa so với mô hình thực được sử dụng để tạo dữ liệu như thế nào? Nhận xét về giá trị hệ số.

- (g) Tạo một đồ thị hiển thị $r_j^2 = 1 - \frac{\text{MSE}_{\text{train}}(\beta_j)}{\text{MSE}_{\text{train}}(\beta^*)}$ cho một phạm vi giá trị trong đó β^* là ước lượng hệ số thứ j cho mô hình tốt nhất chứa j hệ số. Nhận xét về những gì bạn quan sát được. So sánh điều này với đồ thị MSE kiểm định từ (d) như thế nào?

11. Bây giờ chúng ta sẽ thử dự đoán tỷ lệ tội phạm bình quân đầu người dựa trên dữ liệu của Boston bộ.

- (a) Hãy thử một số phương pháp hồi quy được tìm hiểu trong chương này, chẳng hạn như lựa chọn tập con tốt nhất, lasso, hồi quy Ridge và PCR. Trình bày và thảo luận kết quả cho các phương pháp mà bạn đã xem xét.
- (b) Đề xuất một mô hình (hoặc một tập hợp các mô hình) có vẻ hoạt động tốt trên tập dữ liệu này và giải thích lý do. Hãy chắc chắn rằng bạn đang đánh giá hiệu suất của mô hình bằng cách sử dụng lỗi trên tập dữ liệu kiểm định, kiểm định chéo hoặc một phương pháp thay thế hợp lý khác, thay vì sử dụng lỗi trên tập dữ liệu huấn luyện.

288

6. Lựa chọn và điều chỉnh mô hình tuyến tính

(c) Mô hình bạn chọn có bao gồm tất cả các đặc điểm trong dữ liệu không?

Đã thiết lập chưa? Tại sao có hoặc tại sao không?

7



Vượt ra ngoài tính tuyến tính

Cho đến nay trong cuốn sách này, chúng ta chủ yếu tập trung vào các mô hình tuyến tính. Mô hình tuyến tính Chúng tương đối dễ mô tả và triển khai, và có những ưu điểm hơn so với các loại khác. các phương pháp tiếp cận khác về mặt diễn giải và suy luận. Tuy nhiên, hồi quy tuyến tính tiêu chuẩn có thể có những hạn chế đáng kể về khả năng dự đoán. Điều này là do giả định về tính tuyến tính hầu như luôn luôn là một phép tính gần đúng, và đôi khi là một phép tính không chính xác. Trong Chương 6, chúng ta thấy rằng chúng ta có thể Cải thiện phương pháp bình phương tối thiểu bằng cách sử dụng hồi quy Ridge, Lasso, hồi quy thành phần chính và các kỹ thuật khác. Trong bối cảnh đó, sự cải thiện được thu được bằng cách giảm độ phức tạp của mô hình tuyến tính, và do đó sự biến thiên của các ước tính. Nhưng chúng ta vẫn đang sử dụng mô hình tuyến tính, có thể Chỉ có thể cải thiện đến mức đó thôi! Trong chương này, chúng ta nói lỏng giả định về tính tuyến tính. đồng thời vẫn cố gắng duy trì khả năng diễn giải cao nhất có thể. Chúng tôi Thực hiện điều này bằng cách xem xét các mở rộng rất đơn giản của mô hình tuyến tính như hồi quy đa thức và hàm bậc thang, cũng như các phương pháp phức tạp hơn. chẳng hạn như hàm spline, hồi quy cục bộ và mô hình cộng tính tổng quát.

- Hồi quy đa thức mở rộng mô hình tuyến tính bằng cách thêm các biến dự báo bổ sung, thu được bằng cách nâng mỗi biến dự báo ban đầu lên lũy thừa. Ví dụ, hồi quy bậc ba sử dụng ba biến X , X^2 và X^3 . như các biến dự đoán. Phương pháp này cung cấp một cách đơn giản để tạo ra sự phù hợp phi tuyến tính với dữ liệu.
- Hàm bậc thang chia phạm vi của một biến thành K vùng riêng biệt. để tạo ra một biến định tính. Điều này có tác dụng phù hợp. một hàm hằng từng đoạn.
- Hàm spline hồi quy linh hoạt hơn so với đa thức và hàm bậc thang, và trên thực tế là một phần mở rộng của cả hai. Chúng liên quan đến phép chia X được chia thành K vùng riêng biệt. Trong mỗi vùng, một hàm đa thức được điều chỉnh sao cho phù hợp với dữ liệu. Tuy nhiên, các đa thức này...

bị ràng buộc để chúng kết nối liền mạch tại các ranh giới vùng, hoặc các nút thắt. Với điều kiện khoảng thời gian được chia thành đủ các vùng, điều này có thể tạo ra sự vừa vặn cực kỳ linh hoạt.

- Đường cong spline làm mịn tương tự như đường cong spline hồi quy, nhưng xuất hiện trong một tình huống hơi khác một chút. Đường cong spline được làm mịn nhờ việc tối thiểu hóa Tiêu chí tổng bình phương phần dư chịu sự điều chỉnh của hình phạt về độ mịn.
- Hồi quy cục bộ tương tự như hàm spline, nhưng khác biệt ở một điểm quan trọng. Các khu vực được phép chồng lấn lên nhau, và thực tế là chúng chồng lấn rất nhiều. Cách thức suôn sẻ.
- Các mô hình cộng tính tổng quát cho phép chúng ta mở rộng các phương pháp trên thành xử lý nhiều biến dự báo.

Trong các phần 7.1-7.6, chúng tôi trình bày một số phương pháp để mô hình hóa mối quan hệ giữa phản hồi Y và một biến dự báo duy nhất X trong một mô hình linh hoạt theo cách này. Trong Phần 7.7, chúng tôi sẽ chỉ ra rằng các phương pháp này có thể được tích hợp liền mạch để mô hình hóa phản hồi Y như một hàm của một số biến dự báo. $X_1, \dots, X_p$.

7.1 Hồi quy đa thức

Về mặt lịch sử, cách thức tiêu chuẩn để mở rộng hồi quy tuyến tính sang các thiết lập trong Mô hình này, trong đó mối quan hệ giữa các biến dự báo và biến phản hồi là phi tuyến tính, đã được sử dụng để thay thế mô hình tuyến tính tiêu chuẩn.

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

với một hàm đa thức

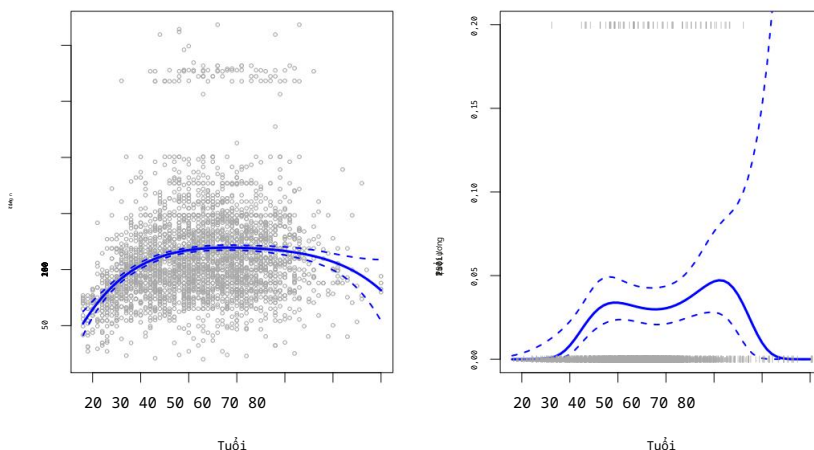
$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_3 x_i^3 + \dots + \beta_d x_i^d + \epsilon_i, \tag{7.1}$$

Trong đó, sai số là gì? Phương pháp này được gọi là hồi quy đa thức, và trên thực tế, chúng ta đã thấy một ví dụ về phương pháp này trong Phần 3.3.2. Đối với các giá trị lớn Với bậc d đủ lớn, hồi quy đa thức cho phép chúng ta tạo ra một kết quả cực kỳ chính xác. Đường cong phi tuyến tính. Lưu ý rằng các hệ số trong (7.1) có thể dễ dàng ước tính sử dụng hồi quy tuyến tính bình phương tối thiểu vì đây chỉ là một phương pháp hồi quy tuyến tính thông thường. mô hình với các biến dự đoán $x_1, x_2, \dots, x_d$. Nói chung, điều đó không phổ biến. nên sử dụng d lớn hơn 3 hoặc 4 vì với các giá trị lớn của d , đa thức Đường cong có thể trở nên quá mềm dẻo và có thể biến dạng thành những hình thù rất kỳ lạ. Điều này đặc biệt đúng gần ranh giới của biến X .

Bảng bên trái trong Hình 7.1 là biểu đồ thể hiện mối quan hệ giữa **tiền lương** và **tuổi tác** của... Bộ dữ liệu **tiền lương**, bao gồm thông tin về thu nhập và nhân khẩu học cho Nam giới cư trú tại khu vực Trung Đại Tây Dương của Hoa Kỳ. Chúng tôi xem kết quả khớp mặt đa thức bậc 4 bằng phương pháp bình phương tối thiểu (đường liền nét) (đường cong màu xanh). Mặc dù đây là mô hình hồi quy tuyến tính giống như bất kỳ mô hình nào khác, Các hệ số riêng lẻ không phải là điều đặc biệt quan tâm. Thay vào đó, chúng ta xem xét Toàn bộ hàm được điều chỉnh phù hợp trên một lưới gồm 63 giá trị **độ tuổi** từ 18 đến 80. để hiểu mối quan hệ giữa **tuổi tác** và **tiền lương**.

đa thức
hồi quy

Đa thức bậc 4



HÌNH 7.1. Dữ liệu về tiền lương. Bên trái: Đường cong màu xanh lam liền nét là một đa thức bậc 4. Mức lương (tính bằng nghìn đô la) là hàm số của độ tuổi, được tính toán bằng phương pháp bình phương nhỏ nhất. Các đường cong nét đứt biểu thị khoảng tin cậy ước tính 95%. Bên phải: Chúng tôi mô hình hóa Sự kiện nhị phân phân mức lương >250 sử dụng hồi quy logistic, một lần nữa với đa thức bậc 4. Xác suất hậu nghiệm ước tính về mức lương vượt quá 250.000 đô la được hiển thị bằng màu xanh lam, dọc theo với khoảng tin cậy ước tính là 95%.

Trong Hình 7.1, một cặp đường cong nét đứt đi kèm với đường cong phù hợp; đây là (2×) Đường cong sai số chuẩn. Hãy xem chúng xuất hiện như thế nào. Giả sử chúng ta đã tính toán được... sự phù hợp tại một giá trị tuổi cụ thể, x_0 :

$$\hat{f}(x_0) = \beta^0 + \beta^1 x_0 + \beta^2 x_0^2 + \beta^3 x_0^3 + \beta^4 x_0^4. \quad (7.2)$$

Phương sai của phép khớp là gì, tức là $\text{Var}[\hat{f}(x_0)]$? Phương pháp bình phương nhỏ nhất trả về phương sai ước tính cho từng hệ số phù hợp β^j , cũng như các hiệp phương sai giữa các cặp ước lượng hệ số. Chúng ta có thể sử dụng chúng để tính toán ước tính phương sai của $\hat{f}(x_0)$. Sai số chuẩn điểm ước tính của $\hat{f}(x_0)$ là căn bậc hai của phương sai này. Phép tính này được lặp lại tại mỗi điểm tham chiếu x_0 , và chúng ta vẽ đường cong phù hợp, cũng như hai lần sai số chuẩn ở hai phía của đường cong phù hợp. Chúng ta vẽ đồ thị hai lần sai số chuẩn vì, đối với các số hạng sai số phân bố chuẩn, đại lượng này Tương ứng với khoảng tin cậy xấp xỉ 95%.

Có vẻ như mức lương trong Hình 7.1 đến từ hai nhóm dân số khác nhau: Đường như có một nhóm người có thu nhập cao, kiếm được hơn 250.000 đô la mỗi năm hàng năm, cũng như nhóm người có thu nhập thấp. Chúng ta có thể coi tiền lương như một biến nhị phân, biến bằng cách chia nó thành hai nhóm này. Sau đó, hồi quy logistic có thể được sử dụng, có thể được sử dụng để dự đoán phân hồi nhị phân này, bằng cách sử dụng các hàm đa thức theo tuổi.

Nếu C^* là ma trận hiệp phương sai 5×5 của β^j và nếu $T_0 = (1, x_0, x_0^2, x_0^3, x_0^4)$, sau đó $\text{Var}[\hat{f}(x_0)] = T_0^T C^* T_0$.

như các biến dự báo. Nói cách khác, chúng ta điều chỉnh mô hình.

$$\Pr(y_i > 250|x_i) = 1 + \frac{\exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_d x_i^d)}{\text{kinh nghiệm}(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_d x_i^d)}.$$

(7.3)

Kết quả được hiển thị ở bảng bên phải của Hình 7.1. Các dấu màu xám
Phản trên và dưới của bảng hiển thị độ tuổi của những người có thu nhập cao.
và những người có thu nhập thấp. Đường cong màu xanh lam liền nét biểu thị xác suất đã được ước tính.
Khả năng có thu nhập cao phụ thuộc vào độ **tuổi**. Khoảng tin cậy ước tính 95% .
Khoảng tin cậy cũng được hiển thị. Chúng ta thấy rằng ở đây các khoảng tin cậy khá chính xác.
Rộng, đặc biệt là ở phía bên phải. Mặc dù kích thước mẫu cho điều này
Bộ dữ liệu khá lớn (n = 3.000), nhưng chỉ có 79 người có thu nhập cao, điều này
Điều này dẫn đến sự biến động lớn trong các hệ số ước tính và do đó,
Khoảng tin cậy rộng.

7.2 Hàm bậc thang

Sử dụng các hàm đa thức của các đặc trưng làm biến dự đoán trong mô hình tuyến tính
áp đặt một cấu trúc toàn cục lên hàm phi tuyến của X. Thay vào đó, chúng ta có thể
Chúng ta sử dụng các hàm bậc thang để tránh áp đặt một cấu trúc toàn cục như vậy. Ở đây,
chúng ta chia phạm vi của X thành các khoảng và gán một hằng số khác nhau cho mỗi khoảng.
Điều này tương đương với việc chuyển đổi một biến liên tục thành một biến phân loại có thứ tự.
biến đổi.

hàm
bậc thang

Cụ thể hơn, chúng ta tạo các điểm cắt c1, c2,...,cK trong phạm vi của X,
và sau đó xây dựng K + 1 biến mới

đã đặt hằng
phân loại
biến

$$\begin{aligned} C_0(X) &= I(X < c_1), \\ C_1(X) &= I(c_1 \leq X < c_2), \\ C_2(X) &= I(c_2 \leq X < c_3), \\ &\vdots \\ C_{K-1}(X) &= I(c_{K-1} \leq X < c_K), \\ C_K(X) &= I(c_K \leq X), \end{aligned}$$

(7.4)

trong đó I(·) là một hàm chỉ thị trả về 1 nếu điều kiện đúng và trả về 0 nếu ngược
lại. Ví dụ, I(cK ≤ X) bằng 1 nếu cK ≤ X, và
Ngược lại, bằng 0. Chúng đôi khi được gọi là biến giả . Lưu ý
rằng với bất kỳ giá trị nào của X, C0(X)+ C1(X)+ ... + CK(X)=1, vì X phải
phải nằm chính xác trong một trong K + 1 khoảng. Sau đó, chúng ta sử dụng phương pháp bình phương tối thiểu để khớp một
mô hình tuyến tính sử dụng C1(X),C2(X),...,CK(X) làm biến dự đoán2:

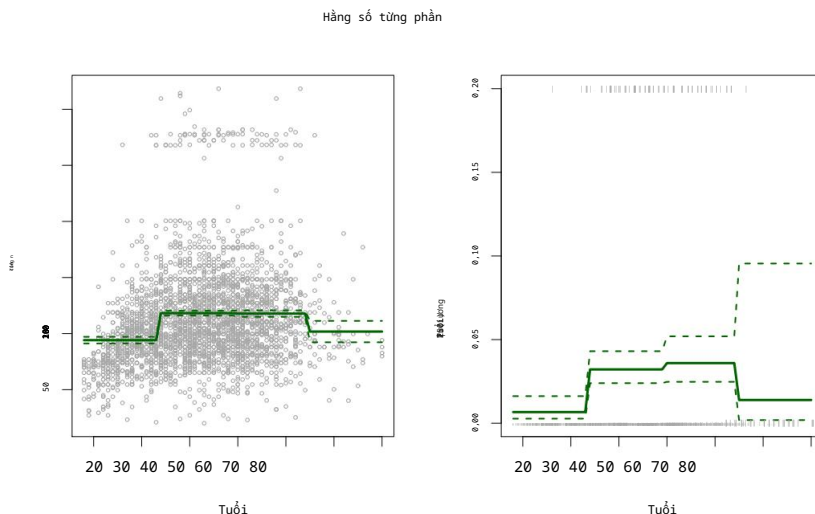
chỉ báo
chức năng

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \dots + \beta_K C_K(x_i) + \epsilon_i.$$

(7.5)

Với một giá trị X cho trước, nhiều nhất chỉ có một trong các C1, C2, ..., CK có thể khác 0.
Lưu ý rằng khi X<c1, tất cả các biến dự đoán trong (7.5) đều bằng 0, do đó β0 có thể

2Chúng tôi loại bỏ C0(X) như một yếu tố dự đoán trong (7.5) vì nó trùng lặp với hệ số chặn.
Điều này tương tự như việc chúng ta chỉ cần hai biến giả để mã hóa một yếu tố định tính.
biến có ba cấp độ, với điều kiện mô hình sẽ chứa một hệ số chặn. Quyết định
việc loại trừ C0(X) thay vì một số Ck(X) khác trong (7.5) là tùy ý. Ngoài ra, chúng ta
có thể bao gồm C0(X),C1(X),...,CK(X) và loại trừ hệ số chặn.



HÌNH 7.2. Dữ liệu về **tiền lương**. Bên trái: Đường cong liên nét hiển thị giá trị được ước tính từ Hồi quy bình phương tối thiểu của **tiền lương** (tính bằng nghìn đô la) sử dụng hàm bậc thang tuổi. Các đường cong nét đứt biểu thị khoảng tin cậy ước tính 95%. Bên phải: Chúng tôi mô hình hóa sự kiện nhị phân **phân mức lương > 250** bằng hồi quy logistic, một lần nữa sử dụng bước hàm số của **tuổi tác**. Xác suất hậu nghiệm ước tính về **mức lương** vượt quá 250.000 đô la là được hiển thị, cùng với khoảng tin cậy ước tính 95%.

được hiểu là giá trị trung bình của Y đối với $X < c_j$. Bằng cách so sánh, (7.5)

Dự đoán phân hồi là $\beta_0 + \beta_j$ cho $c_j \leq X < c_{j+1}$, do đó β_j biểu thị giá trị trung bình.

sự gia tăng phản ứng đối với X trong $c_j \leq X < c_{j+1}$ so với $X < c_1$.

Một ví dụ về việc khớp hàm bậc thang với dữ liệu **Tiền lương** từ Hình 7.1 là:

được hiển thị ở bảng bên trái của Hình 7.2. Chúng tôi cũng đã áp dụng hồi quy logistic. người mẫu

$$\Pr(y_i > 250 | x_i) = 1 + \frac{\exp(\beta_0 + \beta_1 c_1(x_i) + \dots + \beta_K c_K(x_i))}{\exp(\beta_0 + \beta_1 c_1(x_i) + \dots + \beta_K c_K(x_i))} \quad (7.6)$$

để dự đoán xác suất một cá nhân có thu nhập cao trên

dựa trên **độ tuổi**. Bảng bên phải của Hình 7.2 hiển thị phần sau đã được điều chỉnh.

Xác suất thu được bằng phương pháp này.

Đáng tiếc là, trừ khi có những điểm đột phá tự nhiên trong các yếu tố dự đoán, Các hàm hằng từng phần có thể bỏ sót diễn biến. Ví dụ, trong bảng bên trái của Hình 7.2, ô đầu tiên rõ ràng đã bỏ sót xu hướng tăng.

mức lương theo **độ tuổi**. Tuy nhiên, các phương pháp hàm bậc thang rất phổ biến.

trong thống kê sinh học và dịch tễ học, cùng với các lĩnh vực khác. Ví dụ,

Các nhóm tuổi 5 năm thường được sử dụng để xác định các khoảng phân loại.

7.3 Hàm cơ sở

Các mô hình hồi quy đa thức và hằng số từng phần thực chất là những mô hình đặc biệt.

các trường hợp của phương pháp hàm cơ sở. Ý tưởng là có sẵn một họ-

Số lượng các hàm hoặc phép biến đổi có thể áp dụng cho một biến X : $b_1(X), b_2(X), \dots, b_K(X)$. Thay vì sử dụng mô hình tuyến tính cho X , chúng tôi sử dụng mô hình tuyến tính cho X . người mẫu

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \beta_3 b_3(x_i) + \dots + \beta_K b_K(x_i) + \epsilon_i. \tag{7.7}$$

Lưu ý rằng các hàm cơ sở $b_1(\cdot), b_2(\cdot), \dots, b_K(\cdot)$ là cố định và đã biết. (Nói cách khác, chúng ta chọn các hàm số trước.) Đối với đa thức hồi quy, các hàm cơ sở là $b_j(x_i) = x_i^{c_j}$ và đối với hằng số từng phần các hàm chúng là $b_j(x_i) = I(c_j \leq x_i < c_{j+1})$. Chúng ta có thể coi (7.7) như một mô hình tuyến tính tiêu chuẩn với các biến dự đoán $b_1(x_i), b_2(x_i), \dots, b_K(x_i)$. Do đó, Chúng ta có thể sử dụng phương pháp bình phương tối thiểu để ước lượng các hệ số hồi quy chưa biết. trong (7.7). Điều quan trọng là, điều này có nghĩa là tất cả các công cụ suy luận cho tuyến tính các mô hình được thảo luận trong Chương 3, chẳng hạn như sai số chuẩn cho Các ước lượng hệ số và thống kê F để đánh giá ý nghĩa tổng thể của mô hình. có sẵn trong cài đặt này.

Cho đến nay, chúng ta đã xem xét việc sử dụng các hàm đa thức và các hàm hằng từng phần làm hàm cơ sở; tuy nhiên, còn nhiều phương án thay thế khác. Điều này hoàn toàn có thể. Ví dụ, chúng ta có thể sử dụng sóng con hoặc chuỗi Fourier để xây dựng các hàm cơ sở. Trong phần tiếp theo, chúng ta sẽ nghiên cứu một lựa chọn rất phổ biến. Đối với hàm cơ sở: spline hồi quy.

hồi quy
đường cong spline

7.4 Đường cong hồi quy (Regression Splines)

Bây giờ chúng ta sẽ thảo luận về một lớp hàm cơ sở linh hoạt mở rộng dựa trên... các phương pháp hồi quy đa thức và hồi quy hằng số từng phần. Chúng ta vừa mới thấy.

7.4.1 Đa thức từng phần

Thay vì sử dụng đa thức bậc cao để khớp toàn bộ phạm vi của X , hồi quy đa thức từng phần liên quan đến việc khớp các đa thức bậc thấp riêng biệt trên các vùng khác nhau của X . Ví dụ, một đa thức bậc ba từng phần... hoạt động bằng cách áp dụng mô hình hồi quy bậc ba có dạng

từng phần
đa thức
hồi quy

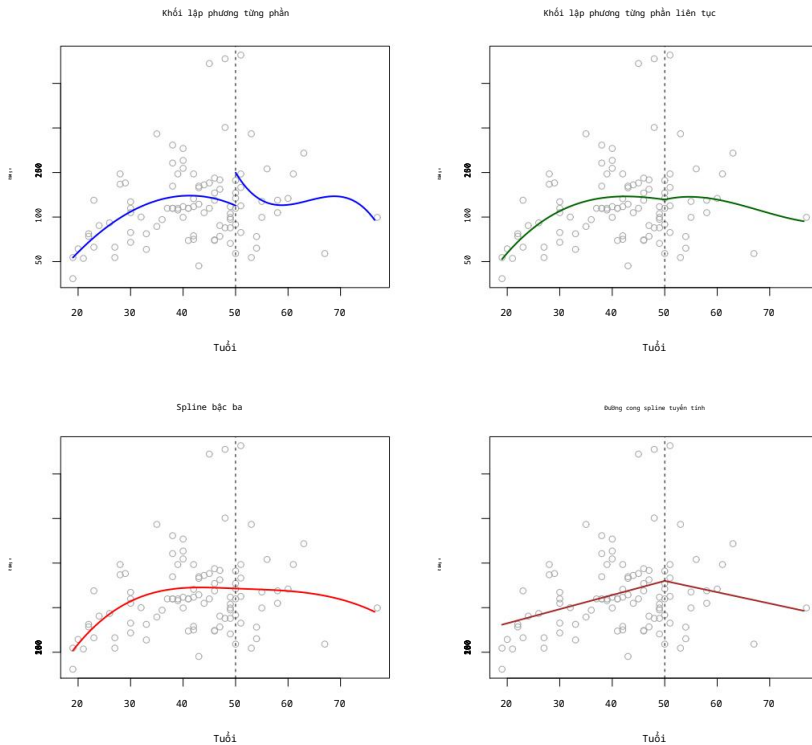
$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_3 x_i^3 + \dots + \epsilon_i, \tag{7.8}$$

trong đó các hệ số $\beta_0, \beta_1, \beta_2$ và β_3 khác nhau ở các phần khác nhau của phạm vi của X . Các điểm mà hệ số thay đổi được gọi là các nút. Ví dụ, một đường cong bậc ba từng phần không có nút thất chỉ là một đường cong bậc ba tiêu chuẩn. đa thức, như trong (7.1) với $d = 3$. Một đa thức bậc ba từng phần với a một nút thất duy nhất tại điểm c có dạng

nút thất

$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \dots + \beta_{31}x_i^3 + \dots + \epsilon_i & \text{nếu } x_i < c \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \dots + \beta_{32}x_i^3 + \dots + \epsilon_i & \text{nếu } x_i \geq c. \end{cases}$$

Nói cách khác, chúng ta sử dụng hai hàm đa thức khác nhau để khớp với dữ liệu, một trong số đó. trên tập con các quan sát có $x_i < c$, và một trên tập con của các quan sát với $x_i \geq c$. Hàm đa thức đầu tiên có các hệ số



HÌNH 7.3. Nhiều đa thức từng phần được điều chỉnh cho phù hợp với một tập con của hàm **tiền lương**. Dữ liệu, với một điểm nút ở **tuổi = 50**. Trên cùng bên trái: Các đa thức bậc ba không bị ràng buộc. Góc trên bên phải: Các đa thức bậc ba được ràng buộc phải liên tục ở **độ tuổi 50**. Góc dưới bên trái: Các đa thức bậc ba bị ràng buộc phải liên tục và phải có đạo hàm bậc nhất và bậc hai liên tục. Góc dưới bên phải: Một đường cong spline tuyến tính được hiển thị, nó bị ràng buộc phải liên tục.

Hệ số thứ nhất là β_{01} , β_{11} , β_{21} và β_{31} , còn hệ số thứ hai là β_{02} , β_{12} , β_{22} và β_{32} . Mỗi hàm đa thức này đều có thể được khớp bằng phương pháp bình phương tối thiểu. thành các hàm đơn giản của bộ dự đoán ban đầu.

Việc sử dụng nhiều nút thắt hơn dẫn đến đa thức từng phần linh hoạt hơn. Nói chung, nếu chúng ta đặt K nút thắt khác nhau trong phạm vi của X , thì chúng ta Cuối cùng sẽ khớp với $K + 1$ đa thức bậc ba khác nhau. Lưu ý rằng chúng ta không cần sử dụng đa thức bậc ba. Ví dụ, thay vào đó chúng ta có thể khớp từng phần các hàm tuyến tính. Trên thực tế, các hàm hằng từng phần của chúng ta trong Phần 7.2 là Đa thức từng phần bậc 0!

Hình 7.3, phần trên bên trái, hiển thị đường cong phù hợp với đa thức bậc ba từng phần. một tập hợp con của dữ liệu **Tiền lương**, với một điểm nút duy nhất ở **độ tuổi 50**. Chúng ta có thể thấy ngay lập tức. Một vấn đề: hàm số không liên tục và trông thật nực cười! Vì mỗi Đa thức có bốn tham số, chúng ta đang sử dụng tổng cộng tám bậc. sự tự do trong việc điều chỉnh mô hình đa thức từng phần này.

mức độ của tự do

7.4.2 Ràng buộc và đường cong Spline

Hình 7.3 , phần trên bên trái, trông có vẻ sai vì đường cong được vẽ khớp chỉ là... quá linh hoạt. Để khắc phục vấn đề này, chúng ta có thể sử dụng đa thức từng phần. với điều kiện là đường cong được điều chỉnh phải liên tục. Trong các trường hợp khác Nói cách khác, không thể có sự nhảy vọt khi $tuổi = 50$. Biểu đồ phía trên bên phải trong Hình 7.3. Hình này thể hiện kết quả khớp dữ liệu. Trông nó tốt hơn hình ở góc trên bên trái, nhưng đường nối hình chữ V trông không tự nhiên.

Trong biểu đồ phía dưới bên trái, chúng tôi đã thêm hai ràng buộc bổ sung: giữ cả hai Đạo hàm bậc nhất và bậc hai của các đa thức từng phần liên tục tại $tuổi = 50$. Nói cách khác, chúng ta đang yêu cầu đa thức từng phần

Không chỉ liên tục khi $tuổi = 50$, mà còn rất mượt mà. Mỗi ràng buộc việc chúng ta áp đặt lên các đa thức bậc ba từng phần về cơ bản giải phóng được một bậc tự do, bằng cách giảm độ phức tạp của kết quả từng phần. khớp đa thức. Vì vậy, trong biểu đồ phía trên bên trái, chúng ta đang sử dụng tám bậc tự do, nhưng trong biểu đồ phía dưới bên trái, chúng ta đã áp đặt ba ràng buộc (liên tục, (Tính liên tục của đạo hàm bậc nhất và tính liên tục của đạo hàm bậc hai) và do đó còn lại năm bậc tự do. Đường cong ở phía dưới bên trái Đồ thị này được gọi là spline bậc ba.³ Nhìn chung, một hàm spline bậc ba với K điểm nút sử dụng tổng cộng $4 + K$ bậc tự do.

Trong Hình 7.3, đồ thị phía dưới bên phải là một đường cong spline tuyến tính, liên tục tại $tuổi = 50$. Định nghĩa chung của một đường cong spline bậc d là một đường cong spline từng phần . Đa thức bậc d, có tính liên tục trong đạo hàm đến bậc $d - 1$ tại mỗi nút. Do đó, một đường cong spline tuyến tính được tạo ra bằng cách khớp một đường thẳng tại mỗi nút. vùng của không gian dự đoán được xác định bởi các nút, đôi khi tính liên tục tại mỗi nút thất.

Trong Hình 7.3, có một nút thất duy nhất ở $tuổi 50$. Tất nhiên, chúng ta có thể thêm vào... thêm nhiều nút thất hơn, và đảm bảo tính liên tục tại mỗi nút.

7.4.3 Biểu diễn cơ sở Spline

Các đường cong hồi quy spline mà chúng ta vừa thấy ở phần trước có thể có Vấn đề có vẻ khá phức tạp: làm thế nào chúng ta có thể khớp một đa thức bậc d từng phần? với điều kiện ràng buộc rằng nó (và có thể là $d - 1$ đạo hàm đầu tiên của nó) phải liên tục? Hóa ra chúng ta có thể sử dụng mô hình cơ sở (7.7) để biểu diễn một hàm spline hồi quy. Một hàm spline bậc ba với K điểm nút có thể được mô hình hóa như sau:

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \dots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i, \tag{7.9}$$

để lựa chọn các hàm cơ sở $b_1, b_2, \dots, b_{K+3}$ một cách thích hợp. Mô hình

(7.9) sau đó có thể được điều chỉnh bằng phương pháp bình phương tối thiểu.

Cũng giống như có nhiều cách để biểu diễn đa thức, cũng có nhiều cách khác nhau. có nhiều cách tương đương để biểu diễn spline bậc ba bằng cách sử dụng các lựa chọn khác nhau của các hàm cơ sở trong (7.9). Cách trực tiếp nhất để biểu diễn spline bậc ba Việc sử dụng (7.9) là bắt đầu với một cơ sở cho đa thức bậc ba—cụ thể là, x, x^2 , và x^3 —và sau đó thêm một hàm cơ sở lũy thừa bị cắt bớt cho mỗi nút.

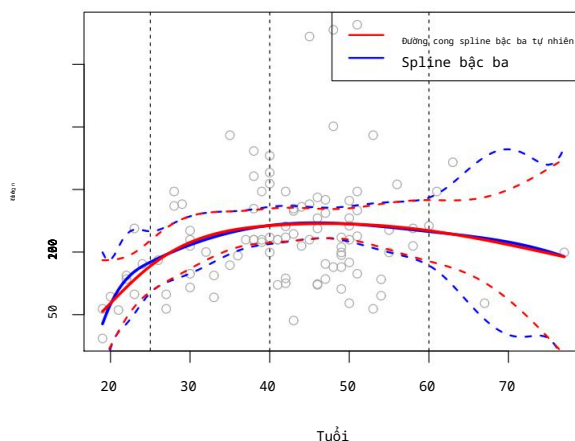
3. Đường cong spline bậc ba phổ biến vì hầu hết mọi người không thể phát hiện ra sự gián đoạn. tại các nút thất.

đạo hàm

đường cong spline bậc ba

đường cong spline tuyến tính

bị cắt ngắn
cơ sở quyền lực



HÌNH 7.4. Một đường cong spline bậc ba và một đường cong spline bậc ba tự nhiên, với ba điểm nút, được điều chỉnh cho một tập hợp con của dữ liệu Tiền lương. Các đường nét đứt biểu thị vị trí các nút thắt.

Hàm cơ sở lũy thừa bị cắt cụt được định nghĩa như sau:

$$h(x, \xi) = \begin{cases} (x - \xi)^3 & \text{nếu } x > \xi \\ 0 & \text{nếu không thì,} \end{cases} \quad (7.10)$$

trong đó ξ là nút thắt. Có thể chứng minh rằng việc thêm một số hạng có dạng $\beta h(x, \xi)$ đối với mô hình (7.8) cho một đa thức bậc ba sẽ dẫn đến sự gián đoạn trong chỉ có đạo hàm bậc ba tại ξ ; hàm số sẽ vẫn liên tục, với Đạo hàm bậc nhất và bậc hai liên tục tại mỗi điểm nút.

Nói cách khác, để khớp một hàm spline bậc ba với một tập dữ liệu có K điểm nút, chúng ta Thực hiện hồi quy bình phương tối thiểu với hệ số chặn và $3 + K$ biến dự đoán, của dạng $X, X_2, X_3, h(X, \xi_1), h(X, \xi_2), \dots, h(X, \xi_K)$, trong đó $\xi_1, \dots, \xi_K$ là các điểm nút. Điều này tương đương với việc ước tính tổng cộng $K + 4$ hệ số hồi quy; vì lý do này, việc khớp một spline bậc ba với K điểm nút sử dụng $K + 4$ bậc của tự do.

Thật không may, hàm spline có thể có độ biến thiên cao ở phạm vi ngoài của nó.

các biến dự báo—tức là, khi X có giá trị rất nhỏ hoặc rất lớn.

giá trị. Hình 7.4 cho thấy sự phù hợp với dữ liệu Tiền lương với ba điểm nút. Chúng ta thấy rằng Các dải tin cậy trong vùng biên có vẻ khá biến động. Spline tự nhiên là spline hồi quy với các ràng buộc biên bổ sung: hàm được yêu cầu phải tuyến tính tại biên (trong vùng mà X là

tự nhiên

đường cong spline

nhỏ hơn nút thắt nhỏ nhất, hoặc lớn hơn nút thắt lớn nhất). Ràng buộc bổ sung này có nghĩa là các đường cong spline tự nhiên thường tạo ra kết quả ổn định hơn.

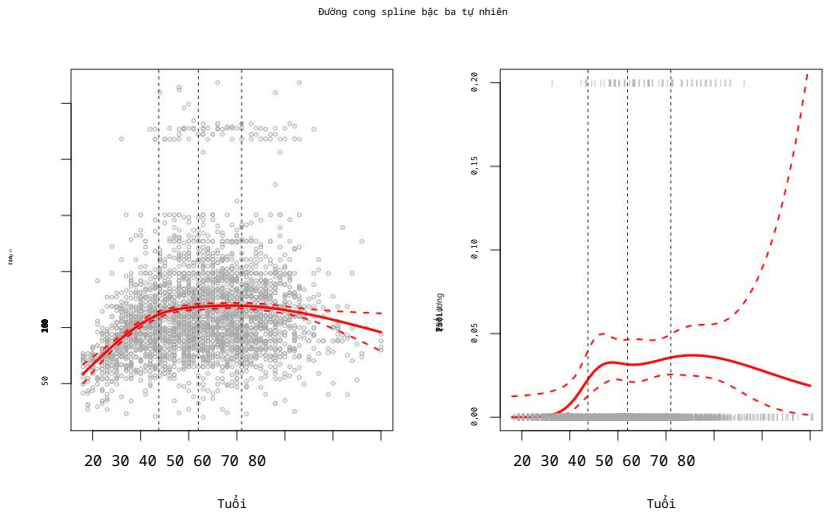
ước tính tại các ranh giới. Trong Hình 7.4, một hàm spline bậc ba tự nhiên cũng được thể hiện. được hiển thị dưới dạng đường màu đỏ. Lưu ý rằng các khoảng tin cậy tương ứng Chúng hẹp hơn.

7.4.4 Lựa chọn số lượng và vị trí các nút thắt

Khi ta sử dụng hàm spline để khớp đường cong, ta nên đặt các điểm nút ở đâu? (Phép hồi quy)

Đường cong spline linh hoạt nhất ở những vùng chứa nhiều điểm nút, bởi vì trong

Những vùng đó, các hệ số đa thức có thể thay đổi nhanh chóng. Do đó, một



HÌNH 7.5. Hàm spline bậc ba tự nhiên với bốn bậc tự do là

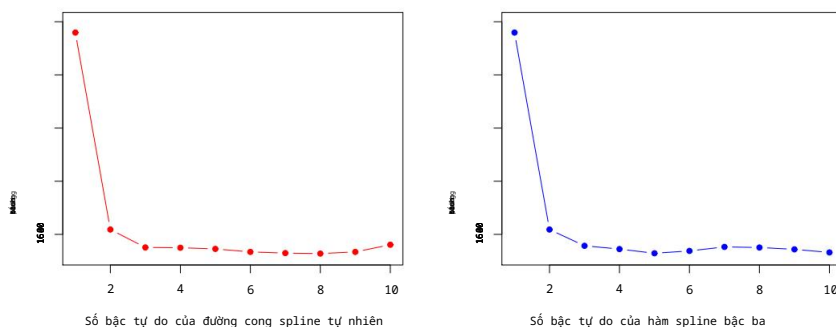
Phù hợp với dữ liệu **Tiền lương**. Bên trái: Một đường cong spline được điều chỉnh phù hợp với **tiền lương** (tính bằng nghìn đô la) như sau: một hàm của **tuổi tác**. Bên phải: Hồi quy logistic được sử dụng để mô hình hóa sự kiện nhị phân. **Mức lương >250** là một hàm số của **tuổi tác**. Xác suất hậu nghiệm ước tính của **mức lương** vượt quá Con số 250.000 đô la được hiển thị. Các đường nét đứt biểu thị vị trí các nút thắt.

Một lựa chọn khác là đặt thêm nhiều nút thắt ở những vị trí mà chúng ta cảm thấy chức năng đó có thể hữu ích. Thay đổi nhanh nhất, và đặt ít nút thắt hơn ở những nơi có vẻ ổn định hơn. Mặc dù phương án này có thể hiệu quả, nhưng trên thực tế, người ta thường thắt nút ở vị trí khác. một cách đồng nhất. Một cách để làm điều này là chỉ định các mức độ mong muốn. sự tự do, và sau đó để phần mềm tự động đặt vị trí tương ứng. Số lượng điểm nút tại các phân vị đồng đều của dữ liệu.

Hình 7.5 minh họa một ví dụ về dữ liệu **Tiền lương**. Tương tự như trong Hình 7.4, chúng ta đã khớp một đường cong spline bậc ba tự nhiên với ba nút, ngoại trừ lần này nút Các vị trí được chọn tự động dựa trên phân vị thứ 25, 50 và 75. về **tuổi**. Điều này được chỉ định bằng cách yêu cầu bốn bậc tự do. Lập luận mà theo đó bốn bậc tự do dẫn đến ba nút thắt bên trong là Hồi mang tính kỹ thuật.4

Chúng ta nên sử dụng bao nhiêu nút thắt, hay nói cách khác là bao nhiêu độ? Đường cong spline của chúng ta nên có độ tự do như thế nào? Một lựa chọn là thử nghiệm với số lượng nút khác nhau và xem số lượng nào tạo ra đường cong đẹp nhất. Một cách nào đó Một cách tiếp cận khác quan trọng hơn là sử dụng phương pháp kiểm định chéo, như đã thảo luận trong Chương 5 và 6. Với phương pháp này, chúng ta loại bỏ một phần dữ liệu (ví dụ 10%), Điều chỉnh đường cong spline với một số điểm nút nhất định cho dữ liệu còn lại, và sau đó Sử dụng hàm spline để đưa ra dự đoán cho phần dữ liệu được giữ lại. Chúng ta lặp lại điều này. lặp lại quy trình nhiều lần cho đến khi mỗi quan sát bị loại bỏ một lần, và

4. Thực tế có năm nút, bao gồm cả hai nút biên. Một spline bậc ba với Năm nút có chín bậc tự do. Nhưng các spline bậc ba tự nhiên có thêm hai bậc tự do nữa. các ràng buộc tự nhiên tại mỗi ranh giới để đảm bảo tính tuyến tính, dẫn đến $9 - 4 = 5$ độ của sự tự do. Vì điều này bao gồm một hằng số, được gộp vào hệ số chặn, nên chúng ta tính toán Nó có bốn bậc tự do.



HÌNH 7.6. Sai số bình phương trung bình được kiểm định chéo mười lần để lựa chọn

bậc tự do khi sử dụng hàm spline để khớp dữ liệu **tiền lương**. Biến phản hồi là **tiền lương**.

và biến dự báo là **tuổi**. Bên trái: Đường cong spline bậc ba tự nhiên. Bên phải: Đường cong spline bậc ba.

Sau đó tính toán RSS tổng thể được kiểm định chéo. Quy trình này có thể được lặp lại cho các số nút K khác nhau. Sau đó, giá trị của K cho kết quả

Nguồn cấp dữ liệu RSS nhỏ nhất được chọn.

Hình 7.6 hiển thị sai số bình phương trung bình được kiểm định chéo mười lần cho các hàm spline. Với nhiều mức độ tự do khác nhau phù hợp với dữ liệu về **tiền lương**. Bảng bên trái Hình bên trái tương ứng với đường cong spline bậc ba tự nhiên, còn hình bên phải tương ứng với đường cong spline bậc ba. Hai phương pháp này cho ra kết quả gần như giống hệt nhau, với sự khác biệt rõ rệt. bằng chứng cho thấy sự phù hợp một bậc (hồi quy tuyến tính) là không đủ. Cả hai các đường cong nhanh chóng trở nên phẳng, và dường như có ba bậc tự do cho Spline tự nhiên và bốn bậc tự do cho spline bậc ba khá giống nhau. đủ.

Trong Phần 7.7, chúng tôi sẽ áp dụng đồng thời các mô hình spline cộng tính trên một số mẫu. các biến tại một thời điểm. Điều này có thể đòi hỏi việc lựa chọn mức độ.

sự tự do cho mỗi biến số. Trong những trường hợp như thế này, chúng ta thường áp dụng một cách tiếp cận khác hơn. áp dụng cách tiếp cận thực dụng và đặt số bậc tự do ở một con số cố định, chẳng hạn như... bốn, cho tất cả các kỹ học.

7.4.5 So sánh với hồi quy đa thức

Hình 7.7 so sánh một hàm spline bậc ba tự nhiên với 15 bậc tự do với một hàm spline khác.

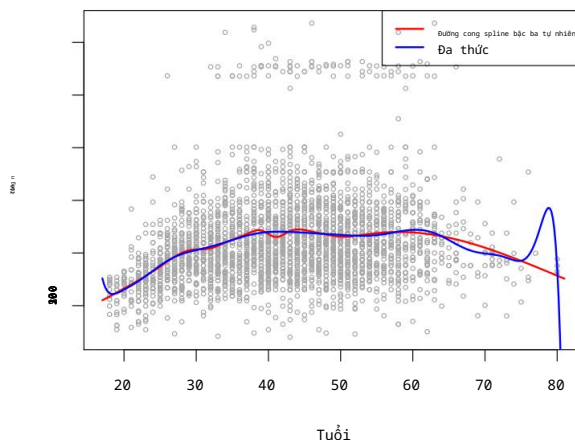
Đa thức bậc 15 trên tập dữ liệu **Wage**. Tính linh hoạt bổ sung trong đa thức tạo ra kết quả không mong muốn ở các ranh giới, trong khi tính tự nhiên

Đường cong spline bậc ba vẫn cho kết quả phù hợp hợp lý với dữ liệu. Đường cong spline hồi quy thường cho kết quả tốt hơn so với hồi quy đa thức. Điều này là do không giống như...

Trong khi các đa thức phải sử dụng bậc cao (số mũ ở hạng tử đơn thức cao nhất, ví dụ:

X^{15}) để tạo ra các đường cong phù hợp linh hoạt, thì hàm spline lại mang đến sự linh hoạt. bằng cách tăng số lượng nút thắt nhưng giữ nguyên độ. Nói chung,

Cách tiếp cận này tạo ra các ước tính ổn định hơn. Hàm spline cũng cho phép chúng ta đặt nhiều nút thắt hơn, và do đó tính linh hoạt cao hơn, trên các vùng mà hàm f có vẻ như vậy. đang thay đổi nhanh chóng, và có ít nút thắt hơn ở những nơi f có vẻ ổn định hơn.



HÌNH 7.7. Trên tập dữ liệu *Tiền lương*, một đường cong spline bậc ba tự nhiên với 15 bậc. Mức độ tự do được so sánh với một đa thức bậc 15. Đa thức có thể thể hiện sự đa dạng phức tạp, hành vi, đặc biệt là gần đuôi.

7.5 Đường cong Spline làm mịn

Trong phần trước, chúng ta đã thảo luận về spline hồi quy, được tạo ra bằng cách xác định một tập hợp các điểm nút, tạo ra một chuỗi các hàm cơ sở, và sau đó sử dụng phương pháp bình phương tối thiểu để ước tính các hệ số spline. Bây giờ chúng ta sẽ giới thiệu một cách tiếp cận hơi khác cũng tạo ra đường cong spline.

7.5.1 Tổng quan về đường cong spline làm mịn

Khi điều chỉnh một đường cong mượt mà cho một tập dữ liệu, điều chúng ta thực sự muốn làm là tìm một hàm nào đó, chẳng hạn như $g(x)$, sao cho phù hợp với dữ liệu quan sát được: nghĩa là, chúng ta muốn $RSS = \sum_{i=1}^N (y_i - g(x_i))^2$ nhỏ. Tuy nhiên, có một vấn đề cách tiếp cận này. Nếu chúng ta không đặt bất kỳ ràng buộc nào lên $g(x_i)$, thì chúng ta có thể luôn luôn đặt RSS bằng 0 chỉ bằng cách chọn g sao cho nó nội suy tất cả của y_i . Một hàm như vậy sẽ làm cho dữ liệu bị khớp quá mức một cách đáng tiếc—nó sẽ xa rời quá linh hoạt. Điều chúng ta thực sự cần là một hàm g giúp thu nhỏ RSS nhưng như vậy cũng trơn tru.

Làm thế nào để đảm bảo g là hàm trơn? Có một số cách để làm điều đó. Hãy làm như vậy. Một cách tiếp cận tự nhiên là tìm hàm g sao cho tối thiểu hóa

$$\sum_{i=1}^N (y_i - g(x_i))^2 + \lambda \int g'(t)^2 dt \quad (7.11)$$

trong đó λ là tham số điều chỉnh không âm. Hàm g là hàm tối thiểu hóa

(7.11) được gọi là spline làm mịn.

(7.11) có nghĩa là gì? Phương trình 7.11 sử dụng công thức “Mất mát + Hình phạt” mà chúng ta gặp trong ngữ cảnh của hồi quy Ridge và Lasso.

Với dữ liệu tốt, và thuật ngữ $\sum_{i=1}^N (y_i - g(x_i))^2$ là hàm mất mát khuyến khích g phù hợp với dữ liệu tốt, và thuật ngữ $\lambda \int g'(t)^2 dt$ là thuật ngữ phạt.

Điều đó trừng phạt sự biến thiên trong g . Ký hiệu $g'(t)$ biểu thị thứ hai

Đạo hàm của hàm g . Đạo hàm bậc nhất $g'(t)$ đo độ dốc

đường cong

spline làm mịn

hàm mất mát

Đạo hàm bậc hai là đạo hàm của một hàm số tại thời điểm t , và đạo hàm bậc hai tương ứng với mức độ thay đổi của độ dốc. Do đó, nói một cách tổng quát, đạo hàm bậc hai của một hàm số là thước đo độ gồ ghề của nó: giá trị tuyệt đối của nó lớn nếu $g(t)$ rất gần ngoèo gần t , và gần bằng 0 trong trường hợp ngược lại. (Đạo hàm bậc hai của một đường thẳng bằng 0; lưu ý rằng một đường thẳng là hoàn toàn trơn tru.)

Ký hiệu này là một tích phân, mà ta có thể coi như là tổng trên phạm vi của t . Nói cách khác, $g(t)^2 dt$ chỉ đơn giản là thước đo tổng biến thiên của hàm $g(t)$ trên toàn bộ phạm vi của nó. Nếu g rất trơn tru, thì $g(t)$ sẽ gần như không đổi và $g(t)^2 dt$ sẽ có giá trị nhỏ.

Ngược lại, nếu g thay đổi thất thường thì $g(t)$ sẽ thay đổi đáng kể và $g(t)^2 dt$ sẽ có giá trị lớn.

Do đó, trong (7.11), $\lambda g(t)^2 dt$ khuyến khích g trở nên trơn tru. Giá trị của λ càng lớn thì g càng trơn tru.

Khi $\lambda = 0$, thì số hạng phạt trong (7.11) không có tác dụng, và do đó hàm g sẽ rất giật cục và sẽ nội suy chính xác các quan sát huấn luyện. Khi $\lambda \rightarrow \infty$, g sẽ hoàn toàn trơn tru – nó sẽ chỉ là một đường thẳng đi qua càng gần các điểm huấn luyện càng tốt.

Trên thực tế, trong trường hợp này, g sẽ là đường thẳng bình phương tối thiểu tuyến tính, vì hàm mất mát trong (7.11) tương đương với việc tối thiểu hóa tổng bình phương dư. Đối với giá trị trung gian của λ , g sẽ xấp xỉ các quan sát huấn luyện nhưng sẽ khá mượt mà. Chúng ta thấy rằng λ kiểm soát sự đánh đổi giữa độ lệch và phương sai của spline làm mịn.

Hàm $g(x)$ làm tối thiểu (7.11) có thể được chứng minh là có một số đặc tính đặc biệt: nó là một đa thức bậc ba từng phần với các nút tại các giá trị duy nhất của $x_1, \dots, x_n$ và đạo hàm bậc nhất và bậc hai liên tục tại mỗi nút. Hơn nữa, nó tuyến tính trong vùng bên ngoài các nút cực trị.

Nói cách khác, hàm $g(x)$ tối thiểu hóa (7.11) là một spline bậc ba tự nhiên với các điểm nút tại $x_1, \dots, x_n$! Tuy nhiên, nó không phải là spline bậc ba tự nhiên giống như spline bậc ba tự nhiên mà ta có được nếu áp dụng phương pháp hàm cơ sở được mô tả trong Phần 7.4.3 với các điểm nút tại $x_1, \dots, x_n$ —mà là một phiên bản thu nhỏ của spline bậc ba tự nhiên đó, trong đó giá trị của tham số điều chỉnh λ trong (7.11) kiểm soát mức độ thu nhỏ.

7.5.2 Lựa chọn tham số làm mịn λ

Chúng ta đã thấy rằng spline làm mịn chỉ đơn giản là một spline bậc ba tự nhiên với các điểm nút tại mỗi giá trị duy nhất của x_i . Có vẻ như spline làm mịn sẽ có quá nhiều bậc tự do, vì một điểm nút tại mỗi điểm dữ liệu cho phép rất nhiều sự linh hoạt. Nhưng tham số điều chỉnh λ kiểm soát độ nhám của spline làm mịn, và do đó là các bậc tự do hiệu quả. Có thể chứng minh rằng khi λ tăng từ 0 đến ∞ , các bậc tự do hiệu quả, mà chúng ta viết là df_{λ} , giảm từ n xuống 2.

hiệu quả
bậc tự do

Trong ngữ cảnh của spline làm mịn, tại sao chúng ta lại thảo luận về bậc tự do hiệu quả thay vì bậc tự do? Thông thường, bậc tự do đề cập đến số lượng tham số tự do, chẳng hạn như số lượng hệ số được đưa vào trong một spline đa thức hoặc spline bậc ba. Mặc dù một spline làm mịn có n tham số và do đó có n bậc tự do danh nghĩa, nhưng n tham số này bị ràng buộc chặt chẽ hoặc bị thu hẹp lại. Do đó, df_{λ} là thước đo tính linh hoạt của spline làm mịn – giá trị càng cao, spline làm mịn càng linh hoạt (và độ lệch càng thấp nhưng phương sai càng cao). Định nghĩa về bậc tự do hiệu quả của

Tự do là một khái niệm khá phức tạp. Chúng ta có thể viết

$$\mathbf{g}^{\lambda} = \mathbf{S}\mathbf{y}, \quad (7.12)$$

trong đó $\mathbf{g}^{\lambda}$ là nghiệm của (7.11) cho một lựa chọn cụ thể của λ —tức là, nó là một vectơ n chiều chứa các giá trị phù hợp của spline làm mịn tại các điểm huấn luyện $x_1, \dots, x_n$. Phương trình 7.12 cho thấy rằng vectơ các giá trị phù hợp khi áp dụng spline làm mịn cho dữ liệu có thể được viết dưới dạng ma trận $\mathbf{S}\lambda \times n$ (có công thức nhân với vectơ phản hồi $\mathbf{y}$). Khi đó, bậc tự do hiệu quả được định nghĩa là

$$df_{\lambda} = \sum_{i=1}^N \{\mathbf{S}\lambda\}_{ii}, \quad (7.13)$$

Tổng các phần tử trên đường chéo chính của ma trận $\mathbf{S}\lambda$.

Trong việc điều chỉnh spline làm mịn, chúng ta không cần phải chọn số lượng hoặc vị trí của các điểm nút—sẽ có một điểm nút tại mỗi quan sát huấn luyện, $x_1, \dots, x_n$. Thay vào đó, chúng ta gặp một vấn đề khác: chúng ta cần chọn giá trị của λ . Không có gì ngạc nhiên khi một giải pháp khả thi cho vấn đề này là kiểm định chéo. Nói cách khác, chúng ta có thể tìm ra giá trị của λ sao cho RSS được kiểm định chéo nhỏ nhất có thể. Hóa ra lỗi kiểm định chéo loại bỏ một phần tử (LOOCV) có thể được tính toán rất hiệu quả đối với spline làm mịn, với chi phí về cơ bản tương đương với việc tính toán một lần điều chỉnh duy nhất, bằng cách sử dụng công thức sau:

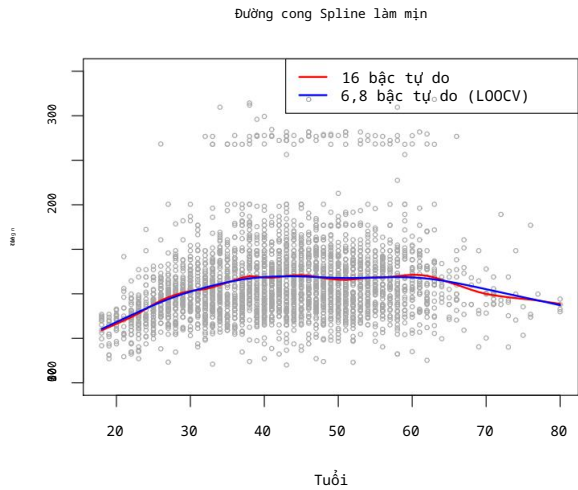
$$RSS_{cv}(\lambda) = \sum_{i=1}^N (y_i - \mathbf{g}^{\lambda}_{(i)}(x_i))^2 = \sum_{i=1}^N \frac{y_i^2 - \mathbf{g}^{\lambda}(x_i)^2}{1 - \{\mathbf{S}\lambda\}_{ii}}.$$

giá tại x_i , trong đó $\mathbf{g}^{\lambda}_{(i)}(x_i)$ biểu thị giá trị phù hợp cho spline làm mịn này. Ký hiệu $\mathbf{g}^{\lambda}$ được đánh pháp phù hợp sử dụng tất cả các quan sát huấn luyện ngoại trừ quan sát thứ i (x_i, y_i). Ngược lại, $\mathbf{g}^{\lambda}(x_i)$ biểu thị hàm spline làm mịn được điều chỉnh cho tất cả các quan sát huấn luyện và được đánh giá tại x_i .

Công thức đáng chú ý này cho biết chúng ta có thể tính toán từng phép khớp loại bỏ một phần tử này chỉ bằng cách sử dụng $\mathbf{g}^{\lambda}$, phép khớp ban đầu với tất cả dữ liệu! Chúng ta có một công thức rất tương tự (5.2) ở trang 205 trong Chương 5 cho hồi quy tuyến tính bình phương nhỏ nhất. Sử dụng (5.2), chúng ta có thể thực hiện LOOCV rất nhanh chóng cho các spline hồi quy đã thảo luận trước đó trong chương này, cũng như cho hồi quy bình phương nhỏ nhất bằng cách sử dụng các hàm cơ sở tùy ý.

Hình 7.8 hiển thị kết quả từ việc khớp một spline làm mịn với dữ liệu **Tiền lương**. Đường cong màu đỏ biểu thị sự khớp thu được khi xác định trước rằng chúng ta muốn một spline làm mịn với 16 bậc tự do hiệu quả. Đường cong màu xanh lam là spline làm mịn thu được khi λ được chọn bằng phương pháp LOOCV; trong trường hợp này, giá trị của λ được chọn dẫn đến 6,8 bậc tự do hiệu quả (được tính bằng (7.13)). Đối với dữ liệu này, có rất ít sự khác biệt đáng kể giữa hai spline làm mịn, ngoài việc spline có 16 bậc tự do có vẻ hơi gợn sóng hơn. Vì có rất ít sự khác biệt giữa hai sự khớp, nên spline làm mịn khớp với 6,8 bậc tự do được chọn.

⁵Các công thức chính xác để tính $\mathbf{g}^{\lambda}(x_i)$ và $\mathbf{S}\lambda$ rất phức tạp về mặt kỹ thuật; tuy nhiên, hiệu quả có sẵn các thuật toán để tính toán các đại lượng này.



HÌNH 7.8. Đường cong spline làm mịn phù hợp với dữ liệu Tiền lương. Đường cong màu đỏ thể hiện kết quả từ việc xác định 16 bậc tự do hiệu quả. Đối với đường cong màu xanh lam, λ đã được tìm thấy được tự động hóa bằng phương pháp kiểm định chéo loại trừ từng phần tử, dẫn đến hiệu quả là 6,8 bậc tự do.

Điều này được ưu tiên hơn, vì nhìn chung các mô hình đơn giản hơn sẽ tốt hơn trừ khi dữ liệu Cung cấp bằng chứng ủng hộ một mô hình phức tạp hơn.

7.6 Hồi quy cục bộ

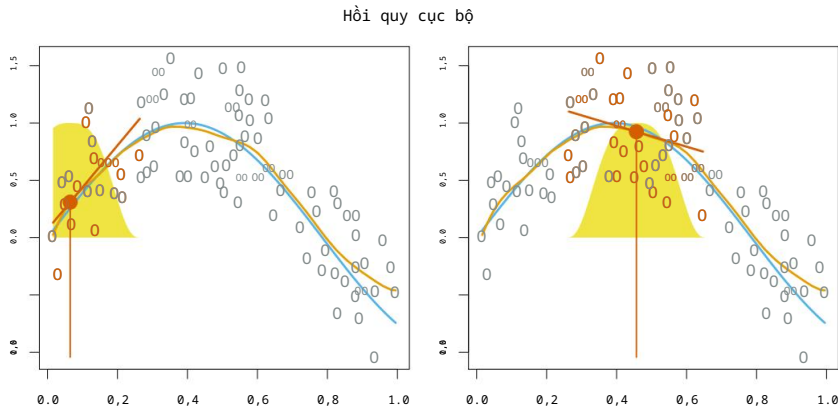
Hồi quy cục bộ là một phương pháp khác để khớp các hàm phi tuyến linh hoạt, bao gồm việc tính toán sự phù hợp tại điểm mục tiêu x_0 chỉ bằng cách sử dụng... các quan sát huấn luyện gần đó. Hình 7.9 minh họa ý tưởng trên một số dữ liệu mô phỏng, với một điểm mục tiêu gần 0,4 và một điểm khác gần ranh giới.

ở mức 0,05. Trong hình này, đường màu xanh biểu thị hàm $f(x)$ từ đó Dữ liệu đã được tạo ra, và đường màu cam nhạt tương ứng với dữ liệu cục bộ. ước tính hồi quy $\hat{f}(x)$. Hồi quy cục bộ được mô tả trong Thuật toán 7.1.

Lưu ý rằng ở Bước 3 của Thuật toán 7.1, trọng số K_{i0} sẽ khác nhau đối với mỗi giá trị của x_0 . Nói cách khác, để có được sự phù hợp hồi quy cục bộ tại một điểm mới là, chúng ta cần xây dựng một mô hình hồi quy bình phương tối thiểu có trọng số mới bằng cách... tối thiểu hóa (7.14) cho một tập hợp trọng số mới. Hồi quy cục bộ đôi khi được gọi là quy trình dựa trên bộ nhớ, bởi vì giống như các láng giềng gần nhất, chúng ta Chúng ta cần tất cả dữ liệu huấn luyện mỗi khi muốn tính toán dự đoán. Tôi sẽ tránh đi sâu vào các chi tiết kỹ thuật về hồi quy cục bộ ở đây. là những cuốn sách được viết về chủ đề này.

Để thực hiện hồi quy cục bộ, có một số lựa chọn. có thể được thực hiện, chẳng hạn như cách định nghĩa hàm trọng số K , và liệu để phù hợp với hồi quy tuyến tính, hằng số hoặc bậc hai ở Bước 3. (Phương trình 7.14) (Tương ứng với hồi quy tuyến tính.) Mặc dù tất cả các lựa chọn này đều có một số sự khác biệt, lựa chọn quan trọng nhất là span s , tức là tỷ lệ. các điểm được sử dụng để tính toán hồi quy cục bộ tại x_0 , như được định nghĩa trong Bước 1. như trên. Khoảng cách đóng vai trò tương tự như tham số điều chỉnh λ trong quá trình làm mịn.

địa phương
hồi quy



HÌNH 7.9. Minh họa hồi quy cục bộ trên một số dữ liệu mô phỏng, trong đó đường cong màu xanh lam biểu thị hàm $f(x)$ được sử dụng để tạo dữ liệu, và đường cong màu cam nhạt tương ứng với ước tính hồi quy cục bộ $\hat{f}(x)$. Các điểm màu cam nằm gần điểm mục tiêu x_0 , được biểu thị bằng đường thẳng đứng màu cam. Hình chuông màu vàng được chồng lên biểu đồ cho biết trọng số được gán cho mỗi điểm, giảm dần về 0 khi khoảng cách từ điểm mục tiêu tăng lên. Giá trị phù hợp $\hat{f}(x_0)$ tại x_0 được thu được bằng cách khớp hồi quy tuyến tính có trọng số (đoạn thẳng màu cam), và sử dụng giá trị phù hợp tại x_0 (chấm tròn màu cam) làm ước tính $\hat{f}(x_0)$.

Tham số s điều chỉnh độ linh hoạt của phép khớp phi tuyến tính. Giá trị s càng nhỏ, phép khớp càng cục bộ và không đều; ngược lại, giá trị s rất lớn sẽ dẫn đến phép khớp toàn cục với dữ liệu bằng cách sử dụng tất cả các quan sát huấn luyện. Chúng ta có thể sử dụng phương pháp kiểm định chéo để chọn s , hoặc chúng ta có thể chỉ định trực tiếp. Hình 7.10 hiển thị các phép khớp hồi quy tuyến tính cục bộ trên dữ liệu Tiền Lương, sử dụng hai giá trị của s : 0,7 và 0,2. Như dự đoán, phép khớp thu được khi sử dụng $s = 0,7$ vượt hơn so với phép khớp thu được khi sử dụng $s = 0,2$.

Khái niệm hồi quy cục bộ có thể được khái quát hóa theo nhiều cách khác nhau. Trong một môi trường có nhiều biến $X_1, X_2, \dots, X_p$, một cách khái quát hóa rất hữu ích là sử dụng mô hình hồi quy tuyến tính đa biến có tính toàn cục đối với một số biến, nhưng lại cục bộ đối với các biến khác, chẳng hạn như thời gian. Các mô hình có hệ số thay đổi như vậy là một cách hữu ích để điều chỉnh mô hình cho phù hợp với dữ liệu được thu thập gần đây nhất. Hồi quy cục bộ cũng được khái quát hóa rất tự nhiên khi chúng ta muốn xây dựng các mô hình cục bộ đối với một cặp biến X_1 và X_2 , thay vì chỉ một biến. Chúng ta có thể đơn giản sử dụng các vùng lân cận hai chiều và áp dụng các mô hình hồi quy tuyến tính hai biến bằng cách sử dụng các quan sát gần mỗi điểm mục tiêu trong không gian hai chiều. Về mặt lý thuyết, phương pháp tương tự có thể được thực hiện trong không gian nhiều chiều hơn, bằng cách sử dụng hồi quy tuyến tính được áp dụng cho các vùng lân cận p chiều. Tuy nhiên, hồi quy cục bộ có thể hoạt động kém hiệu quả nếu p lớn hơn nhiều so với khoảng 3 hoặc 4 vì nhìn chung sẽ có rất ít quan sát huấn luyện gần x_0 . Hồi quy lân cận gần nhất, được thảo luận trong Chương 3, cũng gặp phải vấn đề tương tự trong không gian nhiều chiều.

mô hình hệ
số thay đổi